



SAPIENZA
UNIVERSITÀ DI ROMA

Progettazione e sviluppo di software per validazione esercizi e verifica di codice didattico eseguito su microcontrollore con architettura RISC-V

Facoltà di Ingegneria informatica, Informatica e Statistica
Corso di Laurea in Informatica

Candidato

Carlo Da Roma
Matricola 2086036

Relatore

Prof. Alessandro Checco

Anno Accademico 2025/2026

Progettazione e sviluppo di software per validazione esercizi e verifica di codice didattico eseguito su microcontrollore con architettura RISC-V

Tesi di Laurea. Sapienza – Università di Roma

© 2026 Carlo Da Roma. Tutti i diritti riservati

Questa tesi è stata composta con L^AT_EX e la classe Sapthesis.

Email dell'autore: daromacarlo@gmail.com

Sommario

L'oggetto di questo tirocinio è stata la progettazione e lo sviluppo di un programma per il microcontrollore ESP32-C3 SuperMini (architettura RISC-V), volto ad agevolare l'apprendimento dell'architettura stessa. Il sistema permette la scrittura, compilazione ed esecuzione di codice Assembly direttamente sul microcontrollore. Il software offre varie modalità di utilizzo ed è utile per validare vari tipi di esercizi concepiti per supportare chi desideri approcciare lo studio dell'architettura RISC-V.

Indice

1	Introduzione	1
1.1	Breve introduzione a RISC-V	1
1.2	Perché studiare assembly RISC-V	2
1.3	Importanza dell'esperienza pratica con microcontrollori	2
1.4	Scelta del microcontrollore	2
1.5	Come interfacciarsi col microcontrollore	2
1.6	Come Eseguire il programma	7
1.7	Panoramica del Programma	9
1.7.1	Tabella per riconoscimento errori segnalati dai Led	10
1.8	Impieghi del programma	10
1.9	Contesto, analisi della letteratura corrente e collocazione	11
2	Il programma	12
2.1	Struttura del software	12
2.2	caricatore.py	12
2.3	validator.ino	17
2.3.1	Gestore LED	18
2.3.2	Ricezione e identificazione tipologia input	19
2.3.3	Esecuzione controlli	20
2.4	controlli_statici.S	24
2.4.1	Procedura PRE-POST controlli	24
2.4.2	Ricostruzione Word istruzione	26
2.4.3	Esecuzione Controlli statici	28
2.4.4	Un esempio di controllo statico	29
2.5	controllo_dinamico_loop.S	30
2.5.1	Procedura PRE-POST controllo	31
2.5.2	Inizializzazione contesto	33
2.5.3	Ricostruzione word istruzione	34
2.5.4	Simulazione istruzione	36
2.5.5	simula_B.S	38
2.5.6	simula_R.S	42
2.5.7	simula_S.S	44
2.5.8	simula_I.S	47
2.5.9	simula_J.S	49
2.5.10	simula_U.S	51
2.6	Il diagramma UML di flusso del programma	54
2.7	La repository GitHub 	57
3	Gli esercizi	58
3.1	Tipologia 1.	58
3.2	Tipologia 1. con test multipli.	59
3.3	Tipologia 2.	60
3.4	Tipologia 3.	62
3.5	Tipologia 4.	63
3.6	L'eserciziario	64

4	Conclusioni	65
4.1	Difficoltà incontrate	65
4.2	Sviluppi possibili	65
	Bibliografia	66

Capitolo 1

Introduzione

1.1 Breve introduzione a RISC-V

RISC-V è l'ultima evoluzione delle architetture **R.I.S.C.** (*Reduced Instruction Set Computer*) è nata nel 2010 in California presso l'università di Berkeley e a differenza delle altre architetture presenti ad oggi sul mercato è uno standard open source, questa caratteristica ha modificato profondamente il mercato dei semiconduttori perché i chip RISC-V non sono proprietà di un singolo produttore ma un bene pubblico gestito dalla RISC-V International, più produttori possono produrre e assemblare su diverse schede lo stesso chip se approvati da questo ente.

In ambito accademico, molti corsi universitari che prevedevano l'apprendimento del MIPS32 come linguaggio assembly stanno migrando a RISC-V perché eredita l'eleganza di MIPS ma elimina costi di licenza e alcuni limiti di MIPS; offre una piattaforma più moderna, più etica ed ha alle spalle una community già molto ampia e consolidata ed un largo impiego in ambito industriale.



Figura 1.1. Logo RISC-V

1.2 Perché studiare assembly RISC-V

Lo studio delle architetture e dell'assembly è una componente fondamentale dell'informatica, grazie a questo si apprende a pieno e in maniera approfondita il funzionamento di un calcolatore moderno. Solo studiando linguaggi e concetti così a basso livello si comprende come il software interagisce con l'hardware e la memoria, si apprendono competenze avanzate utili negli ambiti della sicurezza, del reverse engineering e della progettazione del software e si sviluppa un approccio più rigoroso e consapevole alla programmazione.

1.3 Importanza dell'esperienza pratica con microcontrollori

L'apprendimento delle architetture degli elaboratori non può limitarsi al solo studio teorico ma bisogna anche scrivere codice a basso livello per capire a pieno i concetti.

Lavorare con microcontrollori reali consente di osservare concretamente il funzionamento dell'hardware e di interagire con segnali digitali e analogici, tutto ciò permette di consolidare le conoscenze e rende più significativo lo studio della materia

1.4 Scelta del microcontrollore

Per lo sviluppo del progetto è stato scelto il microcontrollore ESP32 in versione "C3 SuperMini" prodotta da Espressif Systems e distribuita da vari fornitori. La scelta di questo dispositivo è motivata da una serie di fattori che lo rendono adatto all'idea del progetto e ad essere usato in laboratori universitari. La scelta è avvenuta a seguito di una fase preliminare di valutazione e confronto tra diverse soluzioni disponibili sul mercato.

- Monta un chip con architettura RISC-V, fondamentale per eseguire programmi assembly RISC-V nativamente.
- È di dimensioni molto ridotte.
- Ha una documentazione molto completa e facile da consultare.
- È compatibile con Arduino IDE e con altri sistemi largamente diffusi, quindi è facile interfacciarsi con essa.
- È molto economica, ha un costo di circa 5€ ad oggi, Se comprata all'ingrosso costa anche meno.
- Offre un'interfaccia di I/O completa, comprendente diverse GPIO, comunicazioni seriali e una porta USB integrata per alimentazione e programmazione; inoltre è dotata di un LED integrato sul GPIO 8, tutte queste caratteristiche sono utili allo scopo del mio progetto.
- È ampiamente utilizzata e supportata da una community particolarmente attiva e consolidata. Questo aspetto rappresenta un vantaggio significativo se si vuole sviluppare software, in quanto garantisce la disponibilità di numerosi esempi, librerie e progetti già sviluppati, facilitando sia la fase di apprendimento sia quella di sviluppo e soprattutto risoluzione dei problemi.

1.5 Come interfacciarsi col microcontrollore

Per sviluppare il codice e flasharlo sulla scheda ho utilizzato Arduino Ide che consiglio in quanto al suo interno sono contenuti tool preinstallati utilissimi per comunicare e gestire queste schede. [Segue una guida passo passo per interfacciarsi al microcontrollore mediante Arduino Ide.](#)

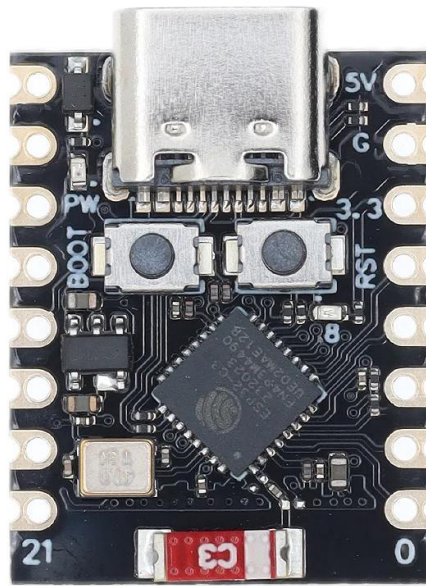


Figura 1.2. ESP32-C3-SuperMini

0) Identificazione parti microcontrollore

Partiamo con l'identificazione di alcune parti della scheda per rendere più agevole la guida.

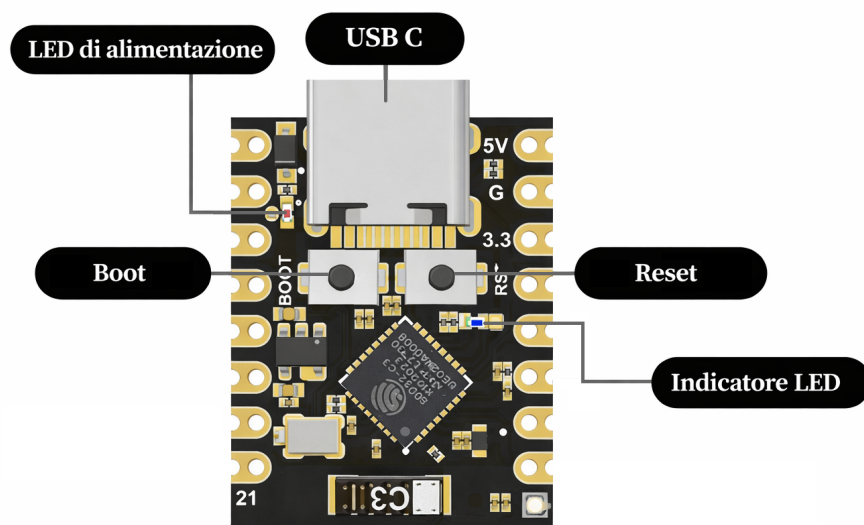


Figura 1.3. ESP32-C3-SuperMini identificazione componenti fondamentali (vista superiore)

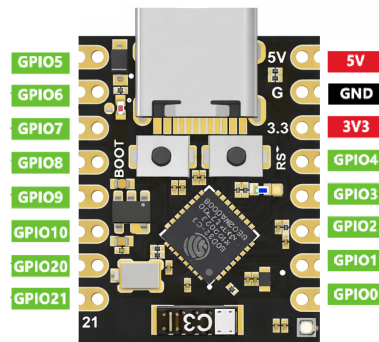


Figura 1.4. ESP32-C3-SuperMini identificazione GPIO (vista superiore)

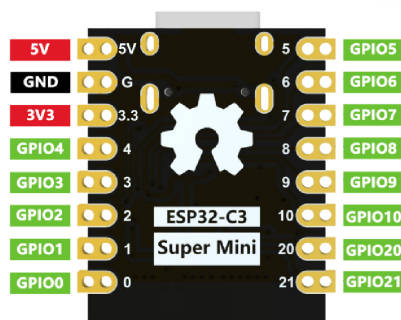


Figura 1.5. ESP32-C3-SuperMini identificazione GPIO (vista inferiore)

Appunto di notazione

D'ora in avanti quando si affermerà che un LED è collegato al pin X , si intende che uno dei terminali di un LED è connesso al GPIO X della scheda, mentre l'altro terminale è collegato al PIN di alimentazione a 3,3V (3V3). In questa configurazione, il GPIO consente di accendere o spegnere il LED tramite segnali logici, mentre il PIN a 3,3V fornisce la tensione necessaria per il corretto funzionamento del componente.

1) Installare Arduino IDE

Scaricare il software dal seguente

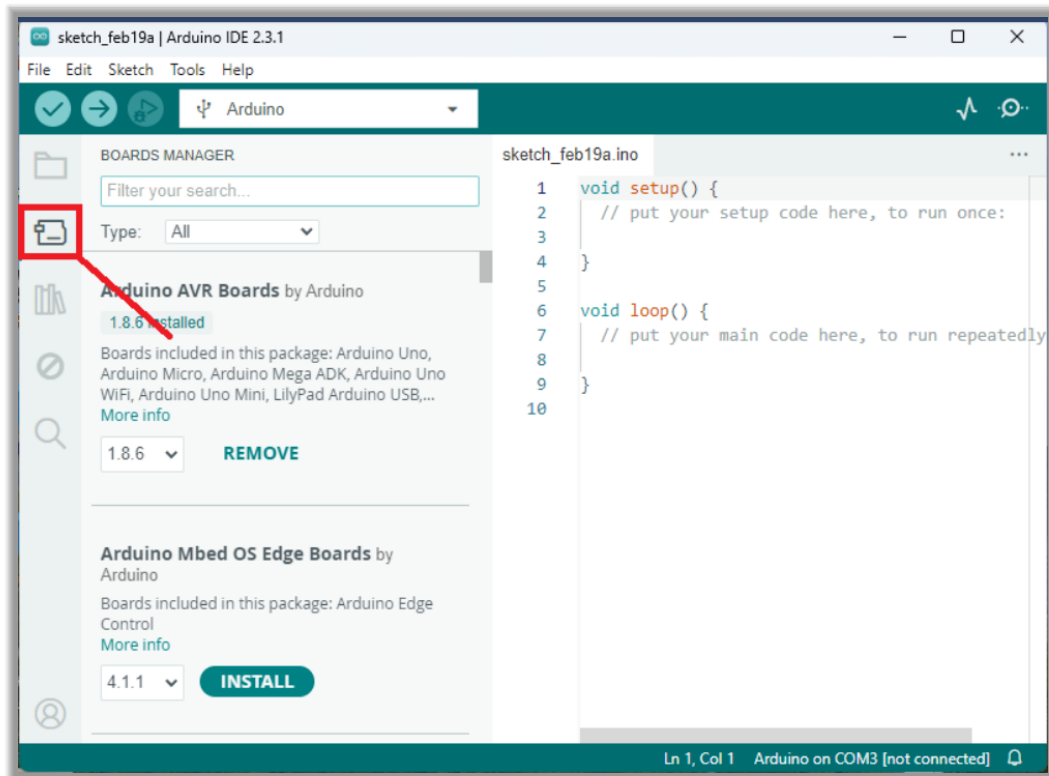
link: <https://www.arduino.cc/en/software/experimental-software>

scegliere la versione più adatta al sistema operativo che avete e completare l'installazione.

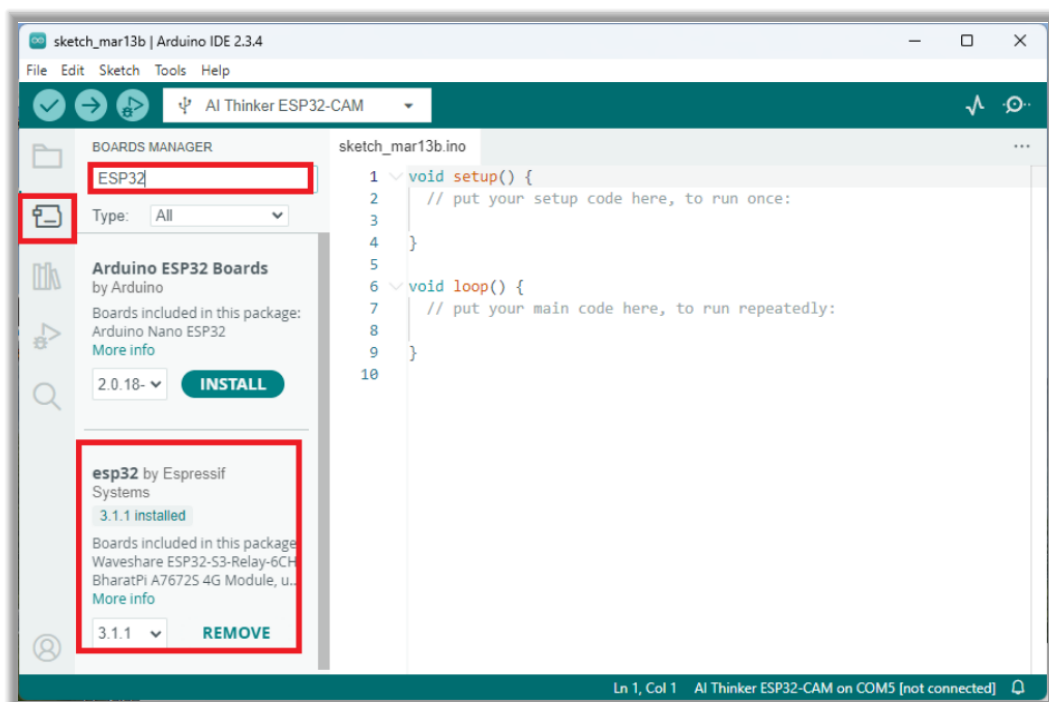
2) Installare i driver della ESP32 in Arduino Ide

clickare l'icona Board Manager (o Gestore Schede in italiano) sul lato sinistro.

In alternativa clickare Tools > Board > BoardManager (Strumenti > Schede > Gestore Schede).



Cercare sulla barra di ricerca "ESP32" ed installare il pacchetto mostrato nella prossima immagine.



Il pacchetto verrà installato in pochi secondi...

3) Collegamento del microcontrollore

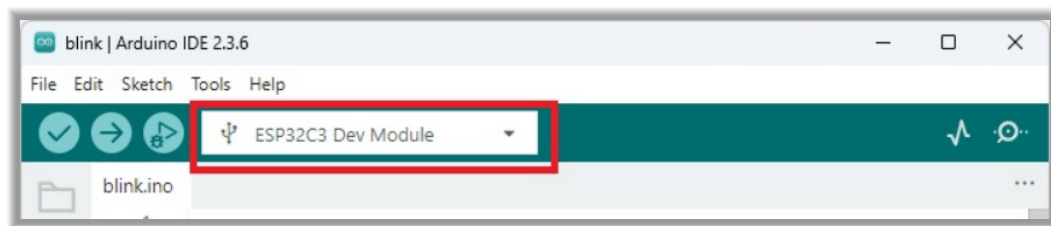
Come già detto la ESP32C3 è dotata di una interfaccia USB-C, nel mio caso specifico per flashare il codice era necessario semplicemente connettere la scheda al computer ma dato che esistono varie versioni di questa scheda potrebbe essere necessario porla in **Bootloader mode**, per impostarla in questa modalità è necessario:

1. Connettere la scheda al computer tramite interfaccia USB-C.
2. Premere il tasto **BOOT**.
3. Mentre si tiene premuto il tasto **BOOT** premere il tasto **RESET** e poi rilasciarlo.
4. Rilasciare il tasto **BOOT**.

4) Caricare codice

Per verificare che tutto sia andato per il meglio eseguiamo codice di prova.

bisogna ora selezionare la scheda e la porta COM per comunicare con l'ESP32.



Cliccare sul menu a tendina in alto e selezionare ESP32C3 Dev Module. Poi, selezionare la porta COM corrispondente (COM3 di solito).

Abilitare poi l'opzione Tools > USB CDC on Boot.

Copiare poi il seguente codice su Arduino IDE. Questo codice fa semplicemente lampeggiare il LED integrato (collegato al PIN 8) e stampa nel Serial Monitor lo stato attuale del PIN.

```

/*****
Codice di prova per far accendere e spegnere
il led integrato sulla scheda in loop
*****/

// il LED integrato dell'ESP32 C3 Super Mini
// funziona con logica invertita
const int ledPin = 8;

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  digitalWrite(ledPin, HIGH);
  Serial.println("LED SPENTO");
  delay(1000);
  digitalWrite(ledPin, LOW);
  Serial.println("LED ACCESO");
  delay(5000);
}

```

Infine, cliccare sul pulsante di Upload (Carica) in alto a sinistra nell'Ide per caricare il codice in memoria . Premi il pulsante **RESET** della scheda per resettarla, toglierla dalla modalità bootloader e far quindi partire il codice.

Il LED integrato blu (indicatore LED) lampeggerà ogni secondo.

1.6 Come Eseguire il programma

Segue una breve guida per eseguire il programma.

1) **Clonare codice.** Innanzitutto bisogna clonare la repository contenente il programma dal seguente link: [Repository](#)



2) Dopo averla clonata accedere al percorso `Tesi-RISC-Visore/Validator`

	.vscode	30/03/2026 22:44	Cartella di file	
	build	30/03/2026 22:44	Cartella di file	
	deprecati	30/03/2026 22:44	Cartella di file	
	test	04/05/2026 15:28	Cartella di file	
	arduino-cli.exe	30/03/2026 18:19	Applicazione	36.873 KB
	caricatore.py	04/05/2026 13:08	File di origine Pyt...	26 KB
	controlli_statici.S	03/04/2026 18:38	File S	3 KB
	controllo_dinamico_loop.S	04/04/2026 13:17	File S	5 KB
	controllo_memoria.S	10/03/2026 15:36	File S	2 KB
	controllo_opcode.S	04/04/2026 12:19	File S	3 KB
	controllo_registri_critici.S	10/03/2026 21:25	File S	2 KB
	controllo_registro_zero.S	31/03/2026 12:15	File S	2 KB
	controllo_salti.S	06/03/2026 03:09	File S	4 KB
	controllo_sintassi.S	10/03/2026 15:37	File S	4 KB
	controllo_word_intere.S	05/03/2026 20:19	File S	1 KB
	rars.jar	23/02/2026 18:52	Executable Jar File	1.817 KB
	riscv1.S	04/05/2026 15:25	File S	1 KB
	riscv1_compiled.txt	04/05/2026 15:25	Documento di testo	1 KB
	simula_B.S	15/04/2026 04:27	File S	3 KB
	simula_I.S	15/04/2026 05:08	File S	7 KB
	simula_J.S	15/04/2026 03:45	File S	2 KB
	simula_R.S	15/04/2026 03:41	File S	4 KB
	simula_S.S	15/04/2026 03:40	File S	2 KB
	simula_U.S	15/04/2026 16:01	File S	2 KB
	test_led.asm	04/05/2026 13:05	File ASM	1 KB
	validator.ino	04/05/2026 15:17	File INO	9 KB
	vuota_registri.S	04/03/2026 00:00	File S	1 KB

3) Flashare poi nella scheda il programma validator.ino controllando il numero di COM con cui la ESP scambia i dati (COM1, COM2, COM3...)



- 4) Aprire `caricatore.py` e modificare la variabile **PORTA** in base alla porta in uso dalla vostra ESP32

```

10
11 # --- CONFIGURAZIONE ---
12 PORTA = 'COM3'
13 BAUD = 115200
14 FQBN = 'esp32:esp32:esp32c3:CDCCOnBoot=cdc'
15

```

Figura 1.6. Righe 11-14 di `caricatore.py`

- 5) Avviare il programma `caricatore.py`

1.7 Panoramica del Programma

Il programma sviluppato accetta in ingresso codice in linguaggio assembly RISC-V oppure una sua rappresentazione già compilata o semplicemente scritta a mano in formato esadecimale little-endian.

Il caricamento del programma avviene tramite una semplice interfaccia grafica realizzata in Python che comunica con il microcontrollore tramite una porta USB. Nel caso in cui l'input sia direttamente il codice esadecimale compilato o scritto a mano, il sistema procede con una serie di controlli sia statici sia dinamici, tutti rilevanti per garantire la correttezza e la sicurezza dell'esecuzione. Anche in questo caso, il codice viene sottoposto a una fase di analisi dinamica mediante una macchina virtuale. Tale macchina virtuale esegue il programma in maniera controllata, invece di utilizzare i registri hardware reali, impiega registri simulati memorizzati in memoria e utilizza una memoria simulata ottenuta spostando l'indirizzo base, *è importante notare che la macchina virtuale simula solo la sezione `.text` del programma; la sezione `.data` viene ignorata. Questa scelta limita i programmi eseguibili alla sola esecuzione di codice che non effettui salti basati su dati caricati dalla memoria, limitazione che risulta comunque adeguata agli scopi del presente lavoro.* Inoltre, è presente un meccanismo di watchdog che impone un limite massimo al numero di istruzioni eseguibili; il superamento di tale soglia comporta l'interruzione dell'esecuzione.

Nel caso in cui l'input sia fornito in assembly RISC-V, esso viene inizialmente assemblato tramite lo strumento RARS, che produce il corrispondente codice macchina in formato esadecimale

little-endian. Durante questa fase vengono effettuati i controlli statici come nel caso in cui venga inviato il codice esadecimale, alcuni dei quali superflui data la compilazione. Rilevante è il controllo degli opcode che verifica che tutte le istruzioni utilizzate siano compatibili con l'architettura ESP32. Successivamente, il codice viene eseguito dalla macchina virtuale prima citata, se tutte le verifiche hanno esito positivo, il codice viene infine eseguito nativamente sul microcontrollore.

In entrambi i casi gli errori vengono segnalati tramite i LED del microcontrollore e sulla console della semplice GUI Python.

1.7.1 Tabella per riconoscimento errori segnalati dai Led

Per eseguire correttamente il programma sopra descritto bisogna dotare la scheda di un LED sul PIN 10 e niente più. Di seguito la tabella per riconoscere la natura dell'errore tramite i LED sul microcontrollore, se l'errore viene riconosciuto in fase di compilazione apparirà solo sulla console della GUI python.

Blink LED	PIN LED	Errore
1 blink lento + molti blink veloci	8 (integrato)	errore opcode
2 blink lenti + molti blink veloci	8 (integrato)	errore word intere
3 blink lenti + molti blink veloci	8 (integrato)	errore sintassi
4 blink lenti + molti blink veloci	8 (integrato)	errore salti
5 blink lenti + molti blink veloci	8 (integrato)	errore scrittura registro zero
6 blink lenti + molti blink veloci	8 (integrato)	errore scrittura su registri critici
7 blink lento + molti blink veloci	8 (integrato)	errore memoria
LOOP MORSE	10	loop infinito

1.8 Impieghi del programma

Oltre ad essere un esempio di programmazione su microcontrollori e quindi essere spunto pratico per implementazioni future, grazie a questo sistema, gli studenti che si stanno avvicinando allo studio di RISC-V possono esercitarsi con tre principali tipologie di attività differenti:

1. **Scrittura di codice in assembly RISC-V:** Si richiede di scrivere un programma senza utilizzare salti condizionati dipendenti da valori caricati in RAM dalla sezione `.data`. Il codice deve gestire l'accensione di LED collegati a specifici pin per un tempo prestabilito: il registro `a0` riceve una maschera di bit per selezionare i pin da attivare (ogni bit a 1 accende il pin corrispondente), mentre il registro `a1` ne definisce la durata in millisecondi. In alternativa, l'esercizio può richiedere di modificare i dati in memoria per evitare che il programma entri in un loop infinito.
2. **Modifica di codice esistente:** viene fornito codice assembly funzionante e si chiede agli utenti di modificarne la sezione `.data` per far accendere un determinato LED per un intervallo di tempo definito, secondo la stessa logica della modalità precedente.
3. **Modifica di codice compilato:** agli utenti viene fornito un file già compilato in formato esadecimale little-endian. Essi devono modificare l'esadecimale per correggere un determinato tipo di errore segnalato dai LED sul microcontrollore, il codice può contenere più errori.
4. **Combinazioni avanzate:** è possibile proporre esercizi che combinano le modalità precedenti, ad esempio modifiche sia al codice assembly sia alla sezione dati per ottenere comportamenti specifici sui LED, oppure prevenire errori e loop nel codice compilato.

Le categorie di esercizio citate saranno approfondite e discusse nello specifico nei capitoli successivi.

1.9 Contesto, analisi della letteratura corrente e collocazione

Evoluzione della Didattica nelle Architetture degli Elaboratori

Nel panorama degli strumenti software rivolti all'insegnamento e all'approfondimento pratico delle architetture degli elaboratori, la letteratura scientifica ha storicamente fatto affidamento su simulatori basati su architetture tradizionali o molto commerciali, come MIPS o x86. In questo contesto si collocano i contributi di Tartaglione [1], incentrato sulla generazione e correzione automatizzata di esercizi complessi relativi ai sistemi di memoria cache per ottimizzare i flussi di valutazione, e di Ziegler [2], il quale ha sviluppato un simulatore MIPS orientato all'analisi visiva e all'apprendimento delle dinamiche interne del pipelining, consentendo agli studenti di osservare i conflitti e i flussi di dati nei cammini di esecuzione.

Negli ultimi anni, tuttavia, si è assistito a una transizione netta verso l'architettura open-source RISC-V per i motivi etici prima citati e perché ormai considerata da Patterson e Hennessy come il nuovo standard di riferimento globale per l'insegnamento dell'hardware sia in ambito accademico che industriale [3]. Questa evoluzione ha spinto la ricerca verso la definizione di nuove metodologie di insegnamento:

1. **Simulatori Software e Piattaforme Web:** Strumenti basati su cloud che consentono di tracciare l'esecuzione delle istruzioni in ambienti virtuali o tramite browser [4], offrendo anche strumenti di simulazione visiva per l'analisi dettagliata del flusso di esecuzione e della pipeline RISC-V [5]. Strumenti efficaci per lo studio iniziale e per superare i vincoli logistici del laboratorio fisico ma tendono ad astrarre eccessivamente l'esperienza dello studente, tenendolo ben lontano dai problemi reali dell'elettronica.
2. **Sistemi Basati su FPGA:** Progetti orientati alla sintesi hardware di core RISC-V didattici personalizzati su logiche programmabili, come il core "Educational RISC-V Micro" per laboratori sintetizzabili [6], o l'implementazione di core custom all'interno di corsi universitari [7]. Questo approccio offre una comprensione profonda della microarchitettura e dei circuiti digitali, ma presenta barriere d'ingresso notevoli, tra cui l'elevato costo delle schede di sviluppo FPGA e la complessità d'uso degli strumenti per interagire con esse.

In generale la letteratura recente evidenzia una sentita necessità di integrare microcontrollori reali ed economici all'interno dei percorsi universitari [8], spostando l'attenzione dai simulatori all'hardware reale.[9].

Collocazione del mio lavoro

Il lavoro di tesi da me proposto è un anello di congiunzione tra i due approcci precedentemente elencati in quanto consente agli studenti di mettere mano su hardware reale ed economico e allo stesso tempo usare uno strumento con barriera di ingresso facilmente oltrepassabile con livello di astrazione modulabile in base alla modalità selezionata. Inoltre essendo il progetto libero rappresenta una base per future modifiche, ampliamenti e di conseguenza nuovi impieghi.

Capitolo 2

Il programma

2.1 Struttura del software

Il software sviluppato per questo progetto è stato scritto per l'80% circa in assembly per RISC-V al fine di valorizzare e approfondire in maniera più significativa lo studio dei linguaggi a basso livello ed è diviso in 2 parti principali: `caricatore.py` e `validator.ino`.

2.2 `caricatore.py`

`Caricatore.py`, Scritto in python, funge da caricatore di programmi RISC-V e digest esadecimali. Il programma comprende:

- Un sistema di compilazione per codice assembly RISC-V (tramite RARS),
- Un'interfaccia per il flashing su scheda ESP32 tramite Arduino CLI, *Questi due software (`Rars` e `Arduino-CLI`) sono posizionati all'interno della cartella per essere invocati dal programma.*
- Una semplice GUI per facilitare la comunicazione con la scheda ESP32
All'avvio del programma `caricatore.py` partirà la semplice GUI da cui sarà possibile selezionare il tipo di caricamento che vogliamo effettuare.

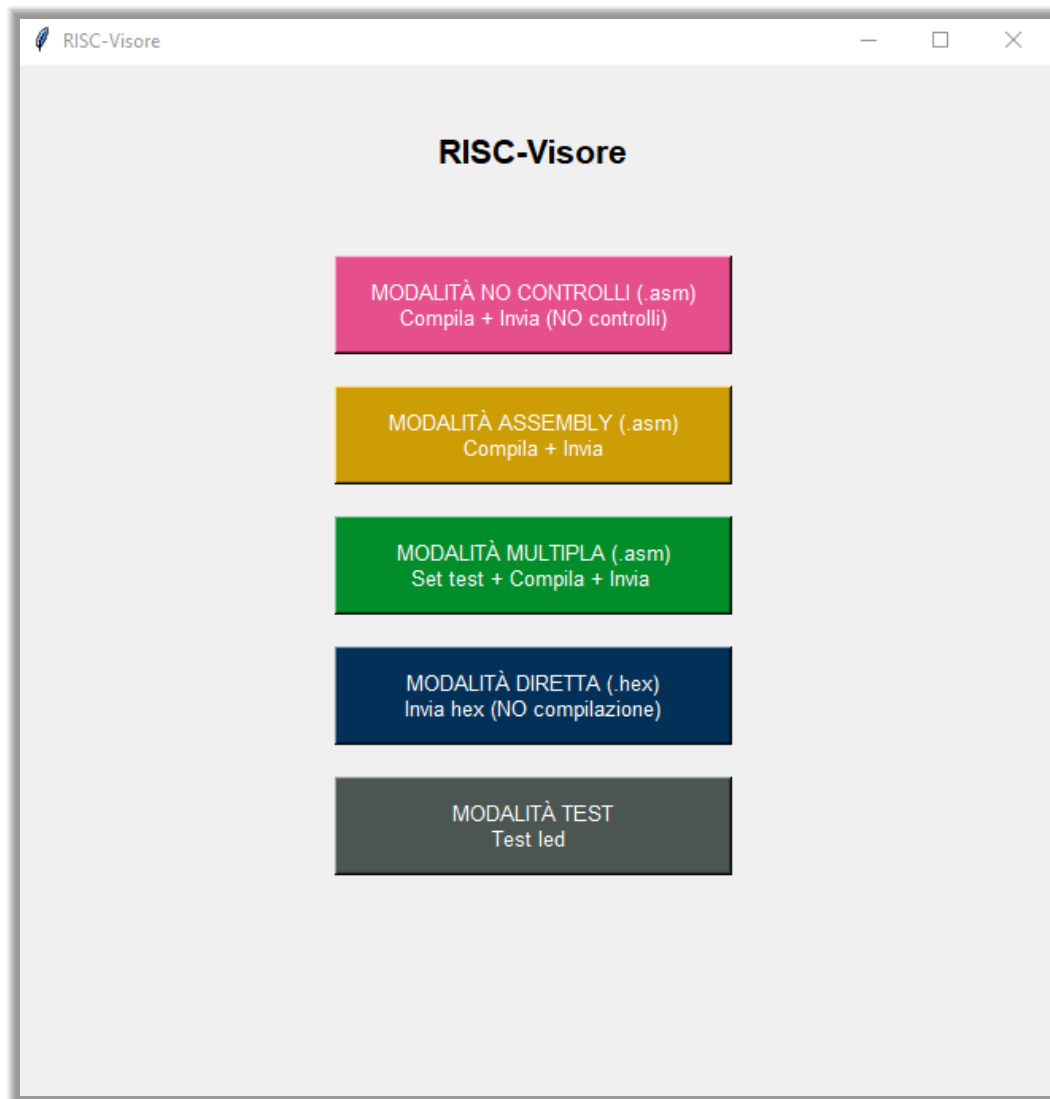


Figura 2.1. Panoramica GUI

- Se viene scelta la **MODALITÀ NO CONTROLLI** il codice caricato viene compilato da Rars e il digest esadecimale creato viene inviato a validator.ino. il programma che deve eseguire la scheda viene ricompilato aggiungendo il codice del programma assembly inviato nel file riscv1.S, il programma inviato verrà eseguito nativamente sulla scheda. **NON VERRANNO ESEGUITI CONTROLLI.**
- Se viene scelta la **MODALITÀ ASSEMBLY** il codice caricato viene compilato da Rars e il digest esadecimale creato viene inviato a validator.ino. il programma che deve eseguire la scheda viene ricompilato aggiungendo il codice del programma assembly inviato nel file riscv1.S, se non si presenteranno errori nelle fasi successive il programma assembly inviato verrà eseguito nativamente sulla scheda.
- Se invece viene selezionata la **MODALITÀ MULTIPLA** il codice caricato viene compilato da Rars e il digest esadecimale creato viene inviato a validator.ino. Insieme a questo in viene chiesto di inserire in input 1 o più .data, il programma che deve eseguire la scheda viene ricompilato aggiungendo il codice del programma assembly inviato nel file riscv1.S, il programma viene eseguito nativamente con tutti i .data inseriti.

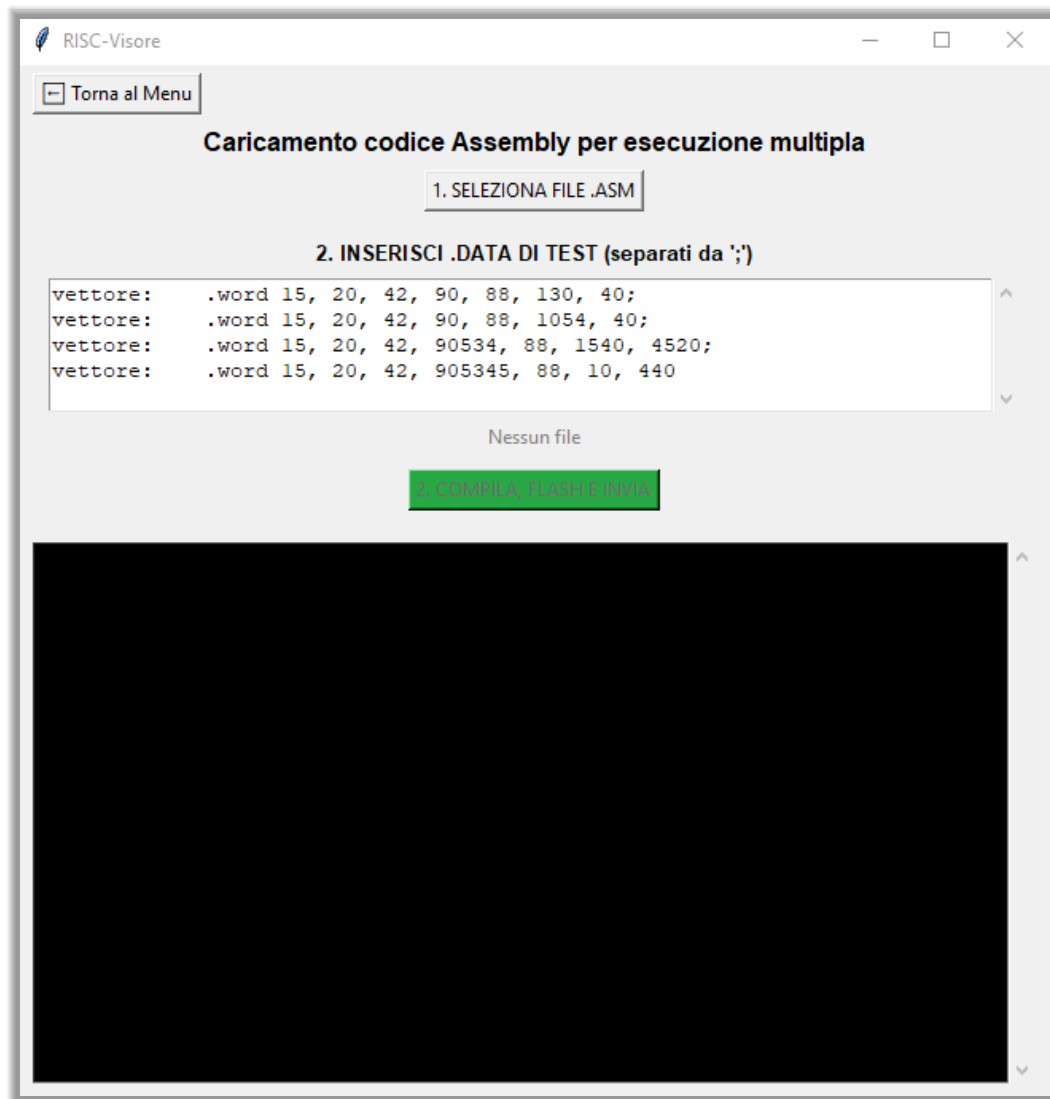


Figura 2.2. Panoramica modalità MULTIPLA

```
=====
ESECUZIONE SET DI TEST 1 DI 2
Payload: vettore:      .word 15, 20, 42, 90, 88, 10, 40
=====
Dati iniettati con successo nella sezione .data esistente
Copia creata: riscv1.S (Label 'main' sostituita con 'riscvprg')

--- Avvio Compilazione e Flash su ESP32-C3 ---
Compilazione validator.ino (attendi qualche istante)...
Compilazione Arduino completata. Avvio caricamento...
Caricamento su ESP32-C3 completato con successo!

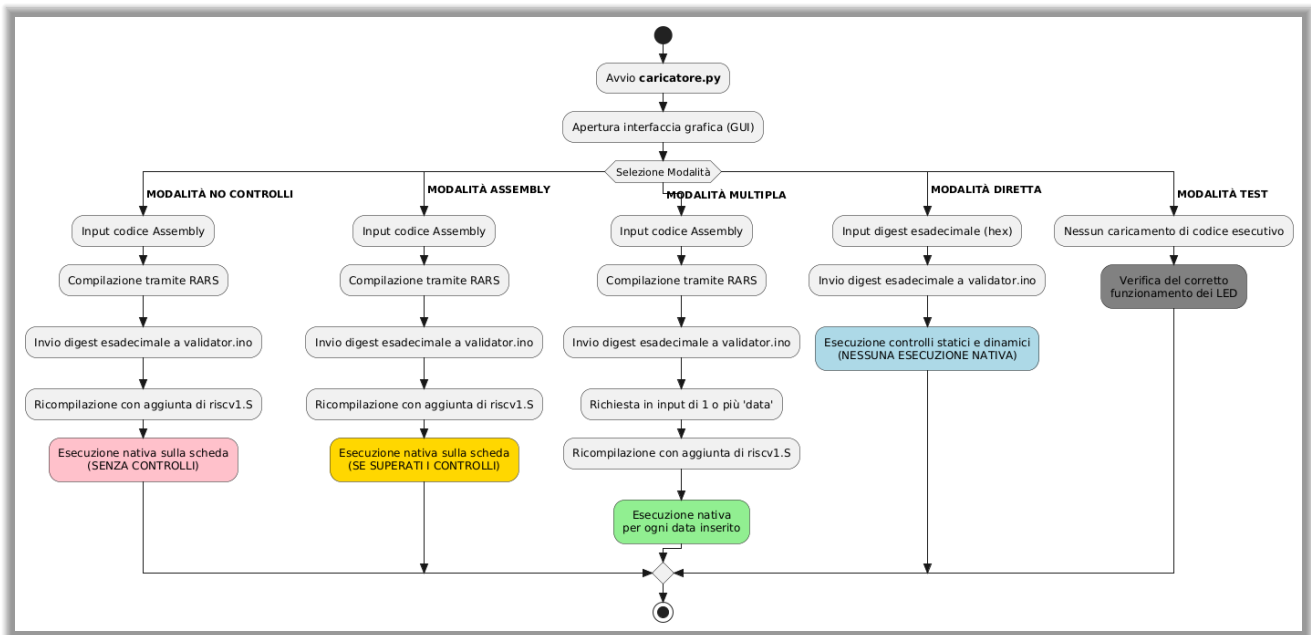
--- Avvio Compilazione RARS ---

Connessione su COM3...
Dati inviati. In attesa di risposta dall'ESP32...
ESP32 > Ricevuti: 144 byte di codice.
ESP32 > Controlli statici superati
ESP32 > Codice funzionante
ESP32 > modalità amu
ESP32 > Esecuzione programma passato...
ESP32 > Tempo: 190 ms, Pin Mask: 1010
--- Sessione conclusa ---

=====
ESECUZIONE SET DI TEST 2 DI 2
Payload: vettore:      .word 20, 14, 5000, 9000, 9312, 100, 50
=====
```

Figura 2.3. Esempio di esecuzione modalità MULTIPLA

- Se invece viene selezionata la [MODALITÀ DIRETTA](#)
Il digest esadecimale caricato viene inviato a validator.ino. per compiere tutti i controlli che verranno descritti nelle sezioni successive, non verrà eseguito nativamente.
- Se viene scelta la [MODALITÀ TEST](#)
Questa procedura non prevede il caricamento di alcun codice esecutivo; il suo scopo è esclusivamente quello di verificare il corretto funzionamento dei LED collegati alla scheda.

Figura 2.4. Diagramma UML di flusso di `caricatore.py`

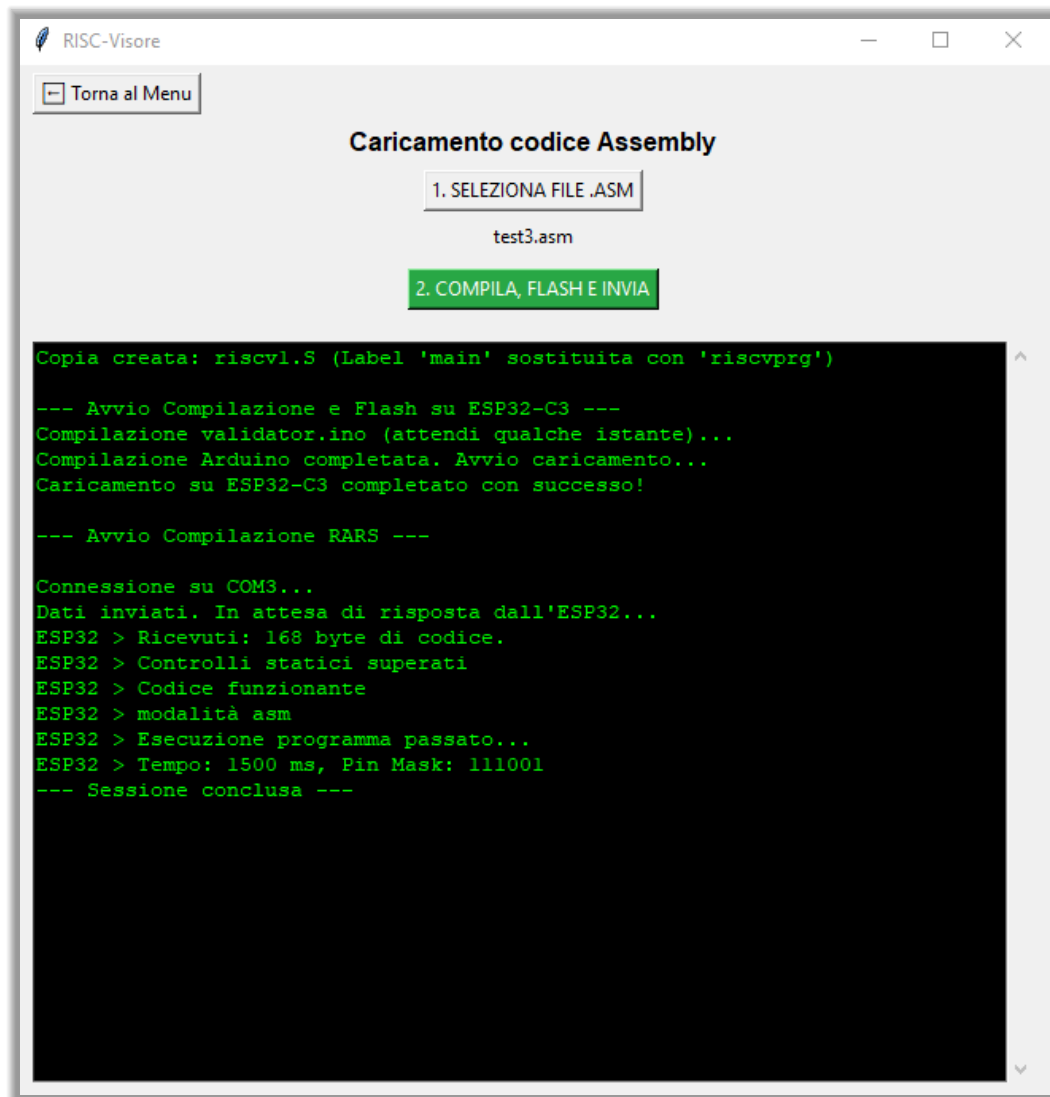


Figura 2.5. Esempio di caricamento

2.3 validator.ino

Validator.ino, scritto in C++, compie diversi compiti, principalmente riceve input da seriale (codice Assembly o digest esadecimale), identifica il tipo di input ed esegue, su di esso, controlli statici e dinamici sul codice passato chiamando le funzioni assembly `controlli_statici.S` e `controllo_dinamico_loop.S` (approfondite nei successivi sottocapitoli) Se il codice è valido per tutti i controlli, viene eseguito (solo nel caso in cui viene passato codice assembly RISC-V non digerito), comprende il gestore dei LED per gestire stati ed errori.

Vediamo ora nel dettaglio i vari segmenti:

2.3.1 Gestore LED

```
29
30 void trasmettiLoopMorse() {
31   const char* morseLoop[] = {".-.-.", "---", "----", "...."};
32   int punto = 200;
33   int linea = 600;
34   for (int i = 0; i < 4; i++) {
35     const char* simbolo = morseLoop[i];
36     for (int j = 0; simbolo[j] != '\0'; j++) {
37       digitalWrite(LED_LOOP_PIN, HIGH);
38       delay(simbolo[j] == '.' ? punto : linea);
39       digitalWrite(LED_LOOP_PIN, LOW);
40       delay(punto);
41     }
42     delay(linea);
43   }
44   Serial.println("controlli dinamici terminati con errore.");
45   Serial.println("RIAVVIA MANUALMENTE LA SCHEDA O ATTENDERE IL RIAVVIO AUTOMATICO.");
46 }
47
48 void gestisciBlink(int n) {
49   if (n <= 0) return;
50   for (int i = 0; i < n; i++) {
51     digitalWrite(LED_PIN, HIGH);
52     delay(600);
53     digitalWrite(LED_PIN, LOW);
54     delay(600);
55   }
56   delay(2000);
57   unsigned long startTime = millis();
58   while (millis() - startTime < 5000) {
59     digitalWrite(LED_PIN, HIGH);
60     delay(100);
61     digitalWrite(LED_PIN, LOW);
62     delay(100);
63   }
64   Serial.println("controlli statici terminati con errore.");
65   Serial.println("RIAVVIA MANUALMENTE LA SCHEDA O ATTENDERE IL RIAVVIO AUTOMATICO.");
66 }
67
```

Figura 2.6. Righe 30-66 di validator.ino

Le funzioni `trasmettiLoopMorse` e `gestisciBlink` sono state implementate per fornire un feedback visivo, da parte dei LED della scheda, degli errori in maniera semplice.

La funzione `gestisciBlink` si occupa di segnalare errori statici rilevati dalla funzione `controlli_statici`. `gestisci blink` ottiene in input un valore `n` da 1 a 7 corrispondente alla tipologia di errore statico e segnala questo con `n` blink lenti e 5 secondi di blink veloci sul LED integrato sul PIN 8.

La funzione `trasmettiLoopMorse`, invece, segnala errori dinamici rilevati durante l'esecuzione della funzione `controlli_dinamico_loop`, il segnale visivo consiste nel LED collegato sul PIN 10 che lampeggia secondo la sequenza del codice Morse corrispondente alla parola "LOOP" ripetuta solo una volta.

```

237 RiscvResult res3 = riscvprg();
238 Serial.print("Tempo: ");
239 Serial.print(res3.tempo);
240 Serial.print(" ms, Pin ");
241 Serial.print("Mask: ");
242 Serial.println(res3.led_pin, BIN);
243 const int mapPin[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 21};
244
245 if (res3.tempo > 0 && res3.led_pin > 0) {
246     for (int i = 0; i < 13; i++) {
247         if (bitRead(res3.led_pin, i)) {
248             pinMode(mapPin[i], OUTPUT);
249             digitalWrite(mapPin[i], LOW);
250         }
251     }
252     delay(res3.tempo);
253     for (int i = 0; i < 13; i++) {
254         if (bitRead(res3.led_pin, i)) {
255             digitalWrite(mapPin[i], HIGH);
256         }
257     }
258 }
259 bufferIndex = 0;
260 return;

```

Figura 2.7. Righe 237-270 di validator.ino (Esempio di gestore led per esercizi)

Questo frammento di codice è rappresentativo della logica di gestione utilizzata per l'attivazione temporizzata dei LED interfacciati al microcontrollore. `Res3.pin_led` è un array di bit in cui ciascun valore identifica lo stato del pin corrispondente: se un bit è impostato a 1 allora il LED associato sarà acceso.

2.3.2 Ricezione e identificazione tipologia input

```

67
68 void loop() {
69     while (Serial.available() > 0) {
70         char c = Serial.read();
71
72         // Accumulo caratteri finché non arrivo a fine riga o riempio il buffer
73         if (c == '\n' || c == '\r') {
74             if (bufferIndex > 0) {
75                 inputBuffer[bufferIndex] = '\0';
76
77                 bool isAsm = (strncmp(inputBuffer, "ASM:", 4) == 0);
78                 bool isHex = (strncmp(inputBuffer, "HEX:", 4) == 0);
79                 bool isAmu = (strncmp(inputBuffer, "AMU:", 4) == 0);
80                 bool isAnc = (strncmp(inputBuffer, "ANC:", 4) == 0);
81                 bool isTst = (strncmp(inputBuffer, "TST:", 4) == 0);
82

```

Figura 2.8. Righe 68-81 di validator.ino

In questa sezione, il codice legge caratteri dalla seriale, costruisce una stringa (`inputBuffer`) e quando riceve `\n` o `\r` considera il messaggio completo, salva la lunghezza dell'input in `datalen`, questa verrà utilizzata dai controlli assembly per capire fin dove finisce il buffer in input. Caricatore.py invia come primi 4 caratteri della stringa un'etichetta che può essere "ASM", "HEX", "AMU", "ANC" o "TST" grazie a questa etichetta `validator.ino` può identificare la tipologia di input e agire di conseguenza.

- ASM identifica un input di tipo assembly.
- HEX identifica un input di tipo esadecimale. digest.

- AMU identifica un input di tipo assembly multiplo.
- ANC identifica un input di tipo assembly al quale non si vogliono applicare controlli.
- TST identifica un input di tipo test.

2.3.3 Esecuzione controlli

```
113
114 // MODALITÀ ANC
115 if (isAnc) {
116     Serial.println("modalità anc, Assembly no controlli");
117     Serial.println("Esecuzione programma passato...");
118     RiscvResult res3 = riscvprg();
119     Serial.print("Tempo: ");
120     Serial.print(res3.tempo);
121     Serial.print(" ms, Pin ");
122     Serial.print("Mask: ");
123     Serial.println(res3.led_pin, BIN);
124     const int mapPin[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 21};
125
126     if (res3.tempo > 0 && res3.led_pin > 0) {
127         for (int i = 0; i < 13; i++) {
128             if (bitRead(res3.led_pin, i)) {
129                 pinMode(mapPin[i], OUTPUT);
130                 digitalWrite(mapPin[i], LOW);
131             }
132         }
133         delay(res3.tempo);
134         for (int i = 0; i < 13; i++) {
135             if (bitRead(res3.led_pin, i)) {
136                 digitalWrite(mapPin[i], HIGH);
137             }
138         }
139     }
140     bufferIndex = 0;
141     return;
142 }
143
```

Figura 2.9. Righe 115-142 di validator.ino

```

143
144 // MODALITÀ ASM
145 if (isAsm) {
146     int res = controllati_statici(dataStart, dataStart + dataLen);
147     if (res == 0) {
148         Serial.println("Controllati statici superati");
149     } else {
150         gestisciBlink(res);
151         bufferIndex = 0;
152         return;
153     }
154     int res2 = controllo_dinamico_loop(dataStart, dataStart + dataLen);
155     if (res2 == 0) {
156         Serial.println("Codice funzionante");
157     }
158     else {
159         Serial.println("Errore dinamico rilevato.");
160         trasmettiLoopMorse();
161         bufferIndex = 0;
162         return;
163     }
164     Serial.println("modalità asm");
165     Serial.println("Esecuzione programma passato...");
166     RiscvResult res3 = riscvprg();
167     Serial.print("Tempo: ");
168     Serial.print(res3.tempo);
169     Serial.print(" ms, Pin ");
170     Serial.print("Mask: ");
171     Serial.println(res3.led_pin, BIN);
172     const int mapPin[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 21};
173
174     if (res3.tempo > 0 && res3.led_pin > 0) {
175         for (int i = 0; i < 13; i++) {
176             if (bitRead(res3.led_pin, i)) {
177                 pinMode(mapPin[i], OUTPUT);
178                 digitalWrite(mapPin[i], LOW);
179             }
180         }
181         delay(res3.tempo);
182         for (int i = 0; i < 13; i++) {
183             if (bitRead(res3.led_pin, i)) {
184                 digitalWrite(mapPin[i], HIGH);
185             }
186         }
187     }
188     bufferIndex = 0;
189     return;
190 }
191

```

Figura 2.10. Righe 145-190 di validator.ino

```
191
192 // MODALITÀ HEX
193 if (isHex) {
194     int res = controllli_statici(dataStart, dataStart + dataLen);
195     if (res == 0) {
196         Serial.println("Controlli statici superati");
197     } else {
198         gestisciBlink(res);
199         bufferIndex = 0;
200         return;
201     }
202     int res2 = controllo_dinamico_loop(dataStart, dataStart + dataLen);
203     if (res2 == 0) {
204         Serial.println("Codice funzionante");
205     } else {
206         Serial.println("Errore dinamico rilevato.");
207         trasmettiLoopMorse();
208         bufferIndex = 0;
209         return;
210     }
211     Serial.println("modalità hex");
212     bufferIndex = 0;
213     return;
214 }
215
```

Figura 2.11. Righe 193-214 di validator.ino

```

215
216 // MODALITÀ AMU
217 if (isAmu) {
218     int res = controllii_statici(dataStart, dataStart + dataLen);
219     if (res == 0) {
220         Serial.println("Controlli statici superati");
221     } else {
222         gestisciBlink(res);
223         bufferIndex = 0;
224         return;
225     }
226     int res2 = controllo_dinamico_loop(dataStart, dataStart + dataLen);
227     if (res2 == 0) {
228         Serial.println("Codice funzionante");
229     } else {
230         Serial.println("Errore dinamico rilevato.");
231         trasmettiLoopMorse();
232         bufferIndex = 0;
233         return;
234     }
235     Serial.println("modalità amu");
236     Serial.println("Esecuzione programma passato...");
237     RiscvResult res3 = riscvprg();
238     Serial.print("Tempo: ");
239     Serial.print(res3.tempo);
240     Serial.print(" ms, Pin ");
241     Serial.print("Mask: ");
242     Serial.println(res3.led_pin, BIN);
243     const int mapPin[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 21};
244
245     if (res3.tempo > 0 && res3.led_pin > 0) {
246         for (int i = 0; i < 13; i++) {
247             if (bitRead(res3.led_pin, i)) {
248                 pinMode(mapPin[i], OUTPUT);
249                 digitalWrite(mapPin[i], LOW);
250             }
251         }
252         delay(res3.tempo);
253         for (int i = 0; i < 13; i++) {
254             if (bitRead(res3.led_pin, i)) {
255                 digitalWrite(mapPin[i], HIGH);
256             }
257         }
258     }
259     bufferIndex = 0;
260     return;
261 }
262
263 bufferIndex = 0;
264 }
265 } else {
266     if (bufferIndex < MAX_BUFFER_SIZE - 1) {
267         inputBuffer[bufferIndex++] = c;
268     }
269 }
270 }
271 }

```

Figura 2.12. Righe 90-126 di validator.ino

In questa sezione, in base al tipo di input, vengono invocate le funzioni assembly `controllii_statici.S` e `controllo_dinamico_loop.S`, incaricate di eseguire i controlli sul codice ricevuto. Eventuali anomalie rilevate, sia in fase di analisi statica sia durante il controllo dinamico, vengono segnalate tramite l'utilizzo dei LED presenti sulla scheda gestiti dal gestore led e tramite la console integrata nella semplice GUI python. Qualora il codice superi con successo tutte le fasi di verifica, si procede con

l'esecuzione del programma caricato. Tale esecuzione avviene solo nel caso in cui l'input fornito sia costituito da codice assembly non precedentemente compilato.

2.4 controlli_statici.S

controlli_statici.S, è un file scritto in Assembly RISC-V, compie diversi controlli sul codice esadecimale che arriva in input da validator.ino. controlli_statici.S è composto da 2 parti principali, una prima parte che ricostruisce le word di istruzioni dall'esadecimale in input e una seconda parte che chiama i controlli su ogni singola word ricostruita.

2.4.1 Procedura PRE-POST controlli

```
5
6  controlli_statici:
7      addi sp, sp, -48
8      sw ra, 44(sp)
9      sw s0, 40(sp)
10     sw s1, 36(sp)
11     sw s2, 32(sp)
12     sw s3, 28(sp)
13     sw s4, 24(sp)
14     sw s9, 20(sp)
15     sw s10, 16(sp)
16     sw s11, 12(sp)
17
```

Figura 2.13. Righe 6-16 di controlli_statici.S

```
111
112 tutto_ok:
113     li a0, 0                # Successo
114     lw ra, 44(sp)
115     lw s0, 40(sp)
116     lw s1, 36(sp)
117     lw s2, 32(sp)
118     lw s3, 28(sp)
119     lw s4, 24(sp)
120     lw s9, 20(sp)
121     lw s10, 16(sp)
122     lw s11, 12(sp)
123     addi sp, sp, 48
124     ret
125
```

Figura 2.14. Righe 112-124 di controlli_statici.S

I registri `sp`, `ra`, `s0`, `s1`, `s2`, `s3`, `s4`, `s9`, `s10`, `s11` sono registri sensibili per le schede della famiglia ESP32 perché contengono informazioni critiche per il corretto funzionamento del contesto di esecuzione del processore, in particolare:

- `sp` (*stack pointer*): punta alla cima dello stack; se viene alterato, si corrompono le chiamate di funzione e il salvataggio dei dati locali.
- `ra` (*return address*): contiene l'indirizzo di ritorno delle funzioni; modificarlo può causare ritorni errati o crash.
- `s0-s11` (*saved registers*): registri calcolati come “callee-saved” secondo la convenzione ABI RISC-V utilizzata dall'ESP32; servono a mantenere lo stato delle funzioni chiamanti. Se vengono modificati senza salvaguardia, si perdono valori importanti tra le chiamate di funzione.

questi registri non vanno quindi sporcati dal codice, per questo vengono salvati i dati contenuti nei registri prima dei controlli nello stack e vengono ricaricati nei registri a fine controlli. **La procedura è fondamentale per la corretta esecuzione del codice.**

2.4.2 Ricostruzione Word istruzione

```
24
25 while:
26     bge t0, s0, tutto_ok    # Scansione byte per byte
27
28     lbu t1, (t0)
29     addi t1, t1, -48
30     ble t1, s1, continua1
31     addi t1, t1, -39
32 continua1:
33     lbu t2, 1(t0)
34     addi t2, t2, -48
35     ble t2, s1, continua2
36     addi t2, t2, -39
37 continua2:
38     lbu t3, 2(t0)
39     addi t3, t3, -48
40     ble t3, s1, continua3
41     addi t3, t3, -39
42 continua3:
43     lbu t4, 3(t0)
44     addi t4, t4, -48
45     ble t4, s1, continua4
46     addi t4, t4, -39
47 continua4:
48     lbu t5, 4(t0)
49     addi t5, t5, -48
50     ble t5, s1, continua5
51     addi t5, t5, -39
52 continua5:
53     lbu s9, 5(t0)
54     addi s9, s9, -48
55     ble s9, s1, continua6
56     addi s9, s9, -39
57 continua6:
58     lbu s10, 6(t0)
59     addi s10, s10, -48
60     ble s10, s1, continua7
61     addi s10, s10, -39
62 continua7:
63     lbu s11, 7(t0)
64     addi s11, s11, -48
65     ble s11, s1, continua8
66     addi s11, s11, -39
67 continua8:
68
```

Figura 2.15. Righe 25-67 di controlli_statici.S

```
68
69      # Ricostruzione Word in t6
70      li t6, 0
71      add t6, t6, s10
72      slli t6, t6, 4
73      add t6, t6, s11
74      slli t6, t6, 4
75      add t6, t6, t5
76      slli t6, t6, 4
77      add t6, t6, s9
78      slli t6, t6, 4
79      add t6, t6, t3
80      slli t6, t6, 4
81      add t6, t6, t4
82      slli t6, t6, 4
83      add t6, t6, t1
84      slli t6, t6, 4
85      add t6, t6, t2
86
```

Figura 2.16. Righe 69-85 di controlli_statici.S

In questa sezione, Il ciclo "while" legge 8 caratteri ASCII esadecimali dall'input per volta. Ogni carattere viene convertito in un nibble (4 bit), e tutti i nibble vengono combinati in una word a 32 bit tramite shift e somme [righe 69-85]. In questo modo, la stringa esadecimale viene trasformata in un valore binario pronto per i controlli. Se il carattere letto in questo ciclo di ricostruzione è un carattere numerico il programma sottrae 48 al valore letto per ottenere il corrispettivo numero, se invece il ciclo incontra una lettera (A-F) sottrae 48 + 39 per trasformarlo nel corrispettivo valore numerico.

Per chiarire meglio il concetto, si riporta la seguente tabella.

b₇ b₆ b₅ b₄ b₃ b₂ b₁ b₀ Bits Column Row					0	1	2	3	4	5	6	7	
					0	1	2	3	4	5	6	7	
0	0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	0	1	0	0	4	EOT	DC4	\$	4	D	T	d	t
0	0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
0	1	0	0	0	8	BS	CAN	(	8	H	X	h	x
0	1	0	0	1	9	HT	EM	)	9	I	Y	i	y
0	1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
0	1	0	1	1	11	VT	ESC	+	;	K	[	k	{
0	1	1	0	0	12	FF	FS	,	<	L	\	l	
0	1	1	0	1	13	CR	GS	—	=	M	]	m	}
0	1	1	1	0	14	SO	RS	.	>	N	^	n	~
0	1	1	1	1	15	SI	US	/	?	O	_	o	DEL

Figura 2.17. Tabella ASCII

2.4.3 Esecuzione Controlli statici

```
86
87     # Chiamata ai validatori
88     jal ra, controllo_opcode
89     jal ra, controllo_memoria
90     jal ra, controllo_registro_zero
91     jal ra, controllo_registri_critici
92     jal ra, controllo_sintassi
93
94     # Controllo Salti
95     andi s3, t6, 127
96     li s2, 99                # Opcode Branch
97     beq s2, s3, vai_controllo_salti
98     li s2, 111              # Opcode JAL
99     beq s2, s3, vai_controllo_salti
100    li s2, 103               # Opcode JALR
101    beq s2, s3, vai_controllo_salti
102    j NO_vai_controllo_salti
103
104    vai_controllo_salti:
105        jal ra, controllo_salti
106
107    NO_vai_controllo_salti:
108        li t6, 0
109        addi t0, t0, 8        # Prossima word
110        j while
111
```

Figura 2.18. Righe 87-110 di controlli_statici.S

In questa sezione vengono chiamati i controlli statici, nell'ordine viene controllato:

1. se l'opcode della word è riconosciuto dall'architettura della ESP32
2. se non ci sono errori in memoria quindi se la word carica o preleva dalla memoria dei dati che sono allineati al byte.
3. se non viene scritto su registro **zero**, viene innanzitutto controllato che non si tratti di una nop word, se la word non è una nop word e il programma tenta di scrivere su **zero** allora viene segnalato un errore.
4. se il programma tenta di scrivere su registri critici
5. se non ci sono errori di sintassi di altra natura
6. se ci sono errori relativi ai salti come ad esempio j fuori programma o j da una istruzione alla stessa.

2.4.4 Un esempio di controllo statico

Data l'impossibilità di esporre esplicitamente in questa trattazione il codice di tutti i controlli propongo quello più significativo, gli altri seguono più o meno la stessa struttura.

```

6
7  controllo_opcode:
8      andi s3,t6,127
9      # controlliamo se l'operazione è di tipo algebrico
10     li s2, 51                                #opcode 0110011
11     beq s2, s3, ok
12
13     # controlliamo se l'operazione è di tipo algebrico immediato
14     li s2, 19                                #opcode 0010011
15     beq s2, s3, ok
16
17     # controlliamo se l'operazione è di tipo caricamento
18     li s2, 3                                 #opcode 0000011
19     beq s2, s3, ok
20
21     # controlliamo se l'operazione è di tipo store
22     li s2, 35                                #opcode 0100011
23     beq s2, s3, ok
24
25     # controlliamo se l'operazione è di tipo branch
26     li s2, 99                                #opcode 1100111
27     beq s2, s3, ok
28
29     # controlliamo se l'operazione è di tipo Jump And Link
30     li s2, 111                                #opcode 1101111
31     beq s2, s3, ok
32
33     # controlliamo se l'operazione è di tipo Jump And Link Reg
34     li s2, 103                                #opcode 1100111
35     beq s2, s3, ok
36
37     # controlliamo se l'operazione è Add Upper Imm to PC
38     li s2, 23                                #opcode 0010111
39     beq s2, s3, ok
40
41     # controlliamo se l'operazione è Load Upper Imm
42     li s2, 55                                #opcode 0110111
43     beq s2, s3, ok
44
45     # controlliamo se l'operazione è una chiamata a sistema
46     li s2, 115                                #opcode 1110011
47     beq s2, s3, ok
48
49     # controlliamo se l'operazione è di tipo fenceS
50     li s2, 15                                #opcode 0001111
51     beq s2, s3, ok
52

```

Figura 2.19. Righe 7-51 di Controllo_opcode.S

In questa sezione alla word in entrata viene applicata una maschera [Riga 8] per esplicitare i bit dell'opcode, i bit ottenuti vengono quindi confrontati con l'insieme degli opcode che il microcontrollore

è in grado di interpretare, per determinare quale istruzione eseguire.

```
55
56     errore:
57         jal ra, vuota_registri
58         li a0,1
59         lw ra, 44(sp)
60         lw s0, 40(sp)
61         lw s1, 36(sp)
62         lw s2, 32(sp)
63         lw s3, 28(sp)
64         lw s4, 24(sp)
65         lw s9, 20(sp)
66         lw s10, 16(sp)
67         lw s11, 12(sp)
68         addi sp, sp, 48
69         ret
70
71         |      |      # se tutto va bene invece:
72
73     ok:
74         li a0, 0      # Opcode valido
75         jr ra
76
```

Figura 2.20. Righe 56-75 di Controllo_opcode.S

Se la word in ingresso ha un opcode che il microcontrollore non riconosce il programma si sposta sull'etichetta di segnalazione errori [Righe 56- 69] viene caricato 1 sul registro `a0` per far compiere al programma `validator.ino` un blink lento e 5 secondi di blink veloci al LED collegato al PIN 8 e viene ripristinato lo stato dei registri critici, altrimenti si torna al programma chiamante senza segnalare errori di alcuna natura.

2.5 controllo_dinamico_loop.S

Il file `controllo_dinamico_loop.S` è scritto in Assembly RISC-V e ha il compito di interpretare l'esadecimale generato passato da `validator.ino`. Il programma funziona come una sorta di macchina virtuale che simula l'esecuzione delle istruzioni contenute nella sezione `.text`. A differenza di un'esecuzione nativa, i registri non vengono utilizzati direttamente: quando è necessario salvare dei valori nei registri, questi vengono memorizzati in una porzione di memoria RAM dedicata (`REGISTRI_VIRTUALI: .space 128`), che emula il comportamento dei registri stessi. Inoltre, i dati che normalmente verrebbero salvati

in memoria vengono salvati in una sezione di memoria con indirizzo spostato, creando così una sorta di memoria virtuale. È fondamentale ribadire che il programma simula solo le istruzioni della sezione `.text`, senza gestire la sezione `.data`, come già descritto nelle sezioni precedenti. All'inizio di questa sezione viene inizializzato un watchdog impostato a 4096 passi (`MAXPASSI .word 4096`) nel caso in cui questa soglia venisse superata il programma segnala l'errore di loop infinito. **Questa procedura non riconosce loop infiniti veri e propri** anche perché per il famoso principio dell'halting problem è impossibile riconoscere un loop infinito, tuttavia, limitando i casi ai soli accademici e didattici porre una soglia anche non molto alta è abbastanza al fine di riconoscere programmi che non terminano.

2.5.1 Procedura PRE-POST controllo

```
11
12  .text
13  controllo_dinamico_loop:
14      addi sp, sp, -48
15      sw ra, 44(sp)
16      sw s0, 40(sp)
17      sw s1, 36(sp)
18      sw s2, 32(sp)
19      sw s3, 28(sp)
20      sw s4, 24(sp)
21      sw s9, 20(sp)
22      sw s10, 16(sp)
23      sw s11, 12(sp)
24
```

Figura 2.21. Righe 13-23 di Controllo_dinamico_loop.S

```

153
154 errore_loop_infinito:
155     li a0, 1                # inSuccesso
156     lw ra, 44(sp)
157     lw s0, 40(sp)
158     lw s1, 36(sp)
159     lw s2, 32(sp)
160     lw s3, 28(sp)
161     lw s4, 24(sp)
162     lw s9, 20(sp)
163     lw s10, 16(sp)
164     lw s11, 12(sp)
165     addi sp, sp, 48
166     ret
167
168 ok:
169     li a0, 0                # Successo
170     lw ra, 44(sp)
171     lw s0, 40(sp)
172     lw s1, 36(sp)
173     lw s2, 32(sp)
174     lw s3, 28(sp)
175     lw s4, 24(sp)
176     lw s9, 20(sp)
177     lw s10, 16(sp)
178     lw s11, 12(sp)
179     addi sp, sp, 48
180     ret
181
182 esci:
183     li a0, 2                # inSuccesso (segnala errore anomalo)
184     lw ra, 44(sp)
185     lw s0, 40(sp)
186     lw s1, 36(sp)
187     lw s2, 32(sp)
188     lw s3, 28(sp)
189     lw s4, 24(sp)
190     lw s9, 20(sp)
191     lw s10, 16(sp)
192     lw s11, 12(sp)
193     addi sp, sp, 48
194     ret
195

```

Figura 2.22. Righe 154-194 di controllo_dinamico_loop.S

I registri `sp`, `ra`, `s0`, `s1`, `s2`, `s3`, `s4`, `s9`, `s10`, `s11` sono registri sensibili per le schede della famiglia ESP32 perché contengono informazioni critiche per il corretto funzionamento del contesto di esecuzione del processore, in particolare:

- `sp` (*stack pointer*): punta alla cima dello stack; se viene alterato, si corrompono le chiamate di funzione e il salvataggio dei dati locali.

- **ra** (*return address*): contiene l'indirizzo di ritorno delle funzioni; modificarlo può causare ritorni errati o crash.
- **s0-s11** (*saved registers*): registri calcolati come "callee-saved" secondo la convenzione ABI RISC-V utilizzata dall'ESP32; servono a mantenere lo stato delle funzioni chiamanti. Se vengono modificati senza salvaguardia, si perdono valori importanti tra le chiamate di funzione.

2.5.2 Inizializzazione contesto

```
1  #include <soc/gpio_reg.h>
2  .globl controllo_dinamico_loop, ok
3  .data
4      .align 4
5      CONTAPASSI:      .word 0
6      MAXPASSI:        .word 4096
7      .align 4
8      REGISTRI_VIRTUALI: .space 128
9      .align 4
10     MEMORIA_VIRTUALE: .space 4096
11
```

Figura 2.23. Righe 1-10 di controllo_dinamico_loop.S

In questa sezione viene inizializzato il contesto del simulatore:

- **CONTAPASSI**: contatore delle operazioni simulate; viene incrementato di 1 ad ogni operazione eseguita.
- **MAXPASSI**: valore soglia massimo per **CONTAPASSI**; il suo superamento genera un errore.
- **REGISTRI_VIRTUALI**: area di memoria riservata alla gestione dei registri virtuali.
- **MEMORIA_VIRTUALE**: area di memoria dedicata alla simulazione della memoria virtuale.

2.5.3 Ricostruzione word istruzione

```
43
44 ciclo_simulazione:
45     while:
46         sw zero, 0(a5)
47         li s1, 15
48         addi s4, s4, 1
49         bge s2, s3, ok
50         bgt s4, s5, errore_loop_infinito
51
52         lbu t1, (s2)
53         addi t1, t1, -48
54         ble t1, s1, continua1
55         addi t1, t1, -39
56     continua1:
57         lbu t2, 1(s2)
58         addi t2, t2, -48
59         ble t2, s1, continua2
60         addi t2, t2, -39
61     continua2:
62         lbu t3, 2(s2)
63         addi t3, t3, -48
64         ble t3, s1, continua3
65         addi t3, t3, -39
66     continua3:
67         lbu t4, 3(s2)
68         addi t4, t4, -48
69         ble t4, s1, continua4
70         addi t4, t4, -39
71     continua4:
72         lbu t5, 4(s2)
73         addi t5, t5, -48
74         ble t5, s1, continua5
75         addi t5, t5, -39
76     continua5:
77         lbu s9, 5(s2)
78         addi s9, s9, -48
79         ble s9, s1, continua6
80         addi s9, s9, -39
81     continua6:
82         lbu s10, 6(s2)
83         addi s10, s10, -48
84         ble s10, s1, continua7
85         addi s10, s10, -39
86     continua7:
87         lbu s11, 7(s2)
88         addi s11, s11, -48
89         ble s11, s1, continua8
90         addi s11, s11, -39
91     continua8:
92
```

Figura 2.24. Righe 44-91 di controllo_dinamico_loop.S

92	
93	li t6, 0
94	add t6, t6, s10
95	slli t6, t6, 4
96	add t6, t6, s11
97	slli t6, t6, 4
98	add t6, t6, t5
99	slli t6, t6, 4
100	add t6, t6, s9
101	slli t6, t6, 4
102	add t6, t6, t3
103	slli t6, t6, 4
104	add t6, t6, t4
105	slli t6, t6, 4
106	add t6, t6, t1
107	slli t6, t6, 4
108	add t6, t6, t2
109	

Figura 2.25. Righe 93-108 di controllo_dinamico_loop.S

La prima parte cruciale in questo processo di simulazione consiste nel prelevare i dati da programma validator.ino e decodificarli per simularli in seguito. I dati arrivano in formato esadecimale in un buffer scritto sul registro a0 e come nella sezione precedente il ciclo "while" legge 8 caratteri ASCII esadecimali dall'input per volta. Ogni carattere viene convertito in un nibble (4 bit), e tutti i nibble vengono combinati in una word a 32 bit tramite shift e somme [righe 93-108]. In questo modo, la stringa esadecimale viene trasformata in un valore binario pronto per essere simulato.

2.5.4 Simulazione istruzione

```
109
110      andi s10, t6, 127          # Estraggo Opcode
111
112      li s11, 99
113      beq s10, s11, vai_simula_B
114      li s11, 51
115      beq s10, s11, vai_simula_R
116      li s11, 35
117      beq s10, s11, vai_simula_S
118      li s11, 19
119      beq s10, s11, vai_simula_I
120      li s11, 3
121      beq s10, s11, vai_simula_I
122      li s11, 103
123      beq s10, s11, vai_simula_I
124      li s11, 115
125      beq s10, s11, vai_simula_I
126      li s11, 111
127      beq s10, s11, vai_simula_J
128      li s11, 55
129      beq s10, s11, vai_simula_U
130      li s11, 23
131      beq s10, s11, vai_simula_U
132      j esci    # in teoria è impossibile che si arrivi a questa parte del codice vista la struttura del mio codice
133
```

Figura 2.26. Righe 110-132 di controllo_dinamico_loop.S

```
133
134     vai_simula_B:
135         jal ra, simula_B
136         j while
137     vai_simula_R:
138         jal ra, simula_R
139         j while
140     vai_simula_S:
141         jal ra, simula_S
142         j while
143     vai_simula_I:
144         jal ra, simula_I
145         j while
146     vai_simula_J:
147         jal ra, simula_J
148         j while
149     vai_simula_U:
150         jal ra, simula_U
151         j while
152
```

Figura 2.27. Righe 134-151 di controllo_dinamico_loop.S

In questa sezione di `controllo_dinamico_loop.S` viene estratto l'opcode [riga 110] dell'istruzione appena costruita, le varie istruzioni `beq` [righe 112-132] funzionano da multiplexer direzionale, in base all'opcode il flusso viene spostato su una determinata etichetta e successivamente avverrà la simulazione vera e propria. Ad esempio, se l'istruzione appena ricostruita è un salto condizionato (tipo B con opcode 99) il multiplexer sposterà il flusso del programma sull'etichetta `vai_simula_B` dopodiché verrà chiamata la funzione `simula_B` che simulerà l'istruzione. Dopo la simulazione `simula_B` torna al chiamante che compie un salto incondizionato all'inizio del ciclo principale (`while`) per procedere al fetch dell'istruzione puntata dal PC simulato [righe 136, 139, 142, 145, 148, 151 (`j while`)].

Quando una istruzione viene passata alle varie funzioni di simulazione, viene divisa in pezzi per capire di che istruzione si tratta, e per distinguerne gli operandi. Per chiarire questo concetto la tabella seguente illustra la suddivisione dei bit per i vari formati:

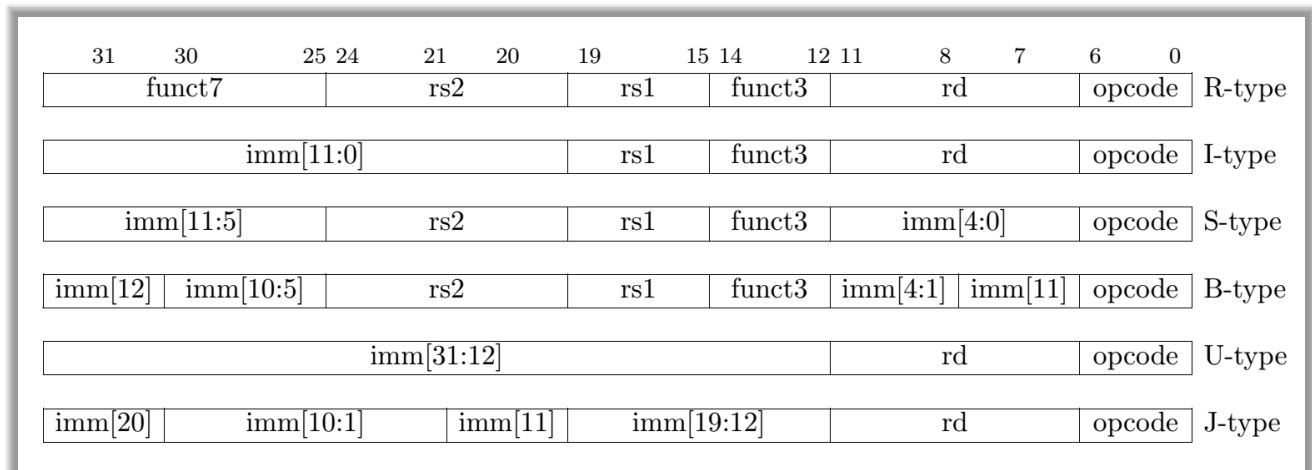


Figura 2.28. set istruzioni RISC-V

Nelle sezioni seguenti viene descritto nello specifico come vengono simulate le istruzioni.

2.5.5 simula_B.S

```

2
3  # IL PROGRAMMA SIMULA LE SEGUENTI OPERAZIONI:
4
5  #-----#
6  # beq Branch == B 1100011 0x0 if(rs1 == rs2) PC += imm
7  # bne Branch != B 1100011 0x1 if(rs1 != rs2) PC += imm
8  # blt Branch < B 1100011 0x4 if(rs1 < rs2)   PC += imm
9  # bge Branch ? B 1100011 0x5 if(rs1 >= rs2)  PC += imm
10 # bltu Branch < (U) B 1100011 0x6 if(rs1 < rs2) PC += imm zero-extends
11 # bgeu Branch ? (U) B 1100011 0x7 if(rs1 >= rs2) PC += imm zero-extends
12 #-----#
13
14 # imm | rs2 | rs1 | funct3 | imm | opcode (B-type)
15

```

Figura 2.29. Righe 3-14 di simula_B.S

Il modulo `simula_B` è responsabile della simulazione delle istruzioni di salto condizionato. Tali istruzioni modificano il flusso di esecuzione del programma aggiornando il PC (Program Counter) sommando il valore del registro PC al campo `imm`, in funzione del risultato del confronto tra i valori contenuti nei registri `rs1` e `rs2`. Possono essere effettuati vari tipi di confronti e in fase di decodifica la tipologia viene dedotta dal valore del campo `funct3`.

```

17
18 .text
19 simula_B:
20     # Estrazione campi rs1, rs2, funct3
21     srli t1, t6, 15
22     andi t1, t1, 31          # t1 = rs1
23
24     srli t2, t6, 20
25     andi t2, t2, 31          # t2 = rs2
26
27     srli t3, t6, 12
28     andi t3, t3, 7           # t3 = funct3
29
30     # Ricostruzione Immediato
31     #-----
32
33     # Bit 12 (segno)
34     srli t4, t6, 31
35     slli t4, t4, 12
36
37     # Bit 11
38     srli t5, t6, 7
39     andi t5, t5, 1
40     slli t5, t5, 11
41
42     # Bit 10:5
43     srli a2, t6, 25
44     andi a2, a2, 0x3F
45     slli a2, a2, 5
46
47     # Bit 4:1
48     srli a3, t6, 8
49     andi a3, a3, 0xF
50     slli a3, a3, 1
51
52     or t4, t4, t5
53     or t4, t4, a2
54     or t4, t4, a3            # t4 contiene l'immediato unito e completo
55
56     slli t4, t4, 19
57     srai t4, t4, 19          # t4 = immediato finale con segno
58

```

Figura 2.30. Righe 18-57 di simula_B.S

Nella prima sezione del codice viene eseguita la decodifica dell'istruzione in ingresso, dalla quale vengono estratti i campi **rs1**, **rs2**, **func3**. In seguito, viene effettuata la ricostruzione del campo imm. dato che in questo formato di istruzioni il campo imm è diviso in più parti nella word d'istruzione vengono estratti separatamente i vari pezzi (bit di segno, bit centrali e bit meno significativi) e successivamente ricostruito combinando questi pezzi con varie operazioni logiche. Una volta ricostruito, l'immediato viene esteso con segno per ottenere il valore finale a 32 bit.

```
60
61     # Carico valori dai registri virtuali
62
63
64     slli t1, t1, 2          # offset rs1
65     add  t1, t1, a5
66     lw   a3, 0(t1)         # a3 = valore rs1
67
68     slli t2, t2, 2          # offset rs2
69     add  t2, t2, a5
70     lw   a4, 0(t2)         # a4 = valore rs2
71
72     # Confronti con func3
73     beq t3, zero, branch_eq # BEQ (funct3=0)
74     li s9, 1
75     beq t3, s9, branch_ne   # BNE (funct3=1)
76     li s9, 4
77     beq t3, s9, branch_lt   # BLT (funct3=4)
78     li s9, 5
79     beq t3, s9, branch_ge   # BGE (funct3=5)
80     li s9, 6
81     beq t3, s9, branch_ltu  # BGE (funct3=6)
82     li s9, 7
83     beq t3, s9, branch_leu  # BGE (funct3=7)
84     jr ra
85
```

Figura 2.31. Righe 61-84 di simula_B.S

Successivamente, i valori contenuti nei registri **rs1** e **rs2** vengono caricati dalla memoria dei registri virtuali. In base al valore di **func3**, il flusso di esecuzione viene indirizzato verso una specifica etichetta, ognuna delle quali implementa una diversa operazione di confronto (uguaglianza, disuguaglianza, minore, maggiore o uguale, sia in versione signed che unsigned).

```
85
86 branch_eq:
87     beq a3, a4, esegui_salto
88     j vai_avanti_con_pc
89 branch_ne:
90     bne a3, a4, esegui_salto
91     j vai_avanti_con_pc
92 branch_lt:
93     blt a3, a4, esegui_salto
94     j vai_avanti_con_pc
95 branch_ge:
96     bge a3, a4, esegui_salto
97     j vai_avanti_con_pc
98 branch_ltu:
99     bltu a3, a4, esegui_salto
100    j vai_avanti_con_pc
101 branch_leu:
102    bgeu a3, a4, esegui_salto
103    j vai_avanti_con_pc
104
105 vai_avanti_con_pc:
106     addi s2, s2, 8
107     jr ra
108
109 esegui_salto:
110     li s11, 2
111     mul t4, t4, s11
112     add s2, s2, t4          # nuovo PC (simulato) = PC (simulato) + immediato
113     jr ra
114
```

Figura 2.32. Righe 86-113 di `simula_B.S`

Infine, viene eseguita la valutazione della condizione: se il confronto ha esito positivo, il PC simulato viene aggiornato sommando l'immediato ($PC = PC + \text{imm}$); altrimenti, il PC simulato avanza di una word per passare all'istruzione successiva. Terminata l'operazione di simulazione, il flusso torna a `controllo_dinamico_loop.S`.

2.5.6 simula_R.S

```

2
3 # IL PROGRAMMA SIMULA LE SEGUENTI OPERAZIONI:
4
5 #-----#
6 # add ADD R 0110011 0x0 0x00 rd = rs1 + rs2
7 # sub SUB R 0110011 0x0 0x20 rd = rs1 - rs2
8 # xor XOR R 0110011 0x4 0x00 rd = rs1 o rs2
9 # or OR R 0110011 0x6 0x00 rd = rs1 | rs2
10 # and AND R 0110011 0x7 0x00 rd = rs1 & rs2
11 # sll Shift Left Logical R 0110011 0x1 0x00 rd = rs1 << rs2
12 # srl Shift Right Logical R 0110011 0x5 0x00 rd = rs1 >> rs2
13 # sra Shift Right Arith* R 0110011 0x5 0x20 rd = rs1 >> rs2 msb-extends
14 # slt Set Less Than R 0110011 0x2 0x00 rd = (rs1 < rs2)?1:0
15 #-----#
16
17 # |funct7 | rs2 | rs1 | funct3 | rd | opcode | (R-type)
18

```

Figura 2.33. Righe 3-17 di simula_R.S

Il modulo `simula_R` è responsabile della simulazione delle istruzioni di tipo R. Queste sono operazioni aritmetico-logiche tra i valori contenuti nei registri `rs1` e `rs2`, producendo un risultato che viene salvato nel registro di destinazione `rd`. La specifica operazione da eseguire è determinata dai campi `func3` e, nei casi necessari, anche dal campo `func7`.

```

25
26     srli t1, t6, 15
27     andi t1, t1, 31          # t1 = rs1
28
29     srli t2, t6, 20
30     andi t2, t2, 31          # t2 = rs2
31
32     srli t3, t6, 12
33     andi t3, t3, 7           # t3 = funct3
34
35     srli t4, t6, 7
36     andi t4, t4, 31          # t4 = rd
37
38     srli t5, t6, 25          # t5 = func7
39     andi t5, t5, 127
40
41
42     # Carico i dati dai registri virtuali (aggiungo l'indirizzo base dei registri virtuali nel calcolo)
43     slli t1, t1, 2           # offset rs1
44     add t1, t1, a5
45     lw a3, 0(t1)            # a3 = valore rs1
46
47     slli t2, t2, 2           # offset rs2
48     add t2, t2, a5
49     lw a4, 0(t2)            # a4 = valore rs2
50
51
52     # Confronti con funct3
53     beq t3, zero, ADD_SUB    # ADD/SUB (funct3=0)
54     li t6, 1
55     beq t3, t6, SLL          # SLL (funct3=1)
56     li t6, 2
57     beq t3, t5, SLT          # SLT (funct3=2)
58     li t6, 3
59     beq t3, t6, SLTU         # SLTU (funct3=3)
60     li t6, 4
61     beq t3, t6, XOR          # XOR (funct3=4)
62     li t6, 5
63     beq t3, t6, SRL_SRA      # SRL/SRA (funct3=5)
64     li t6, 6
65     beq t3, t6, OR           # OR (funct3=6)
66     li t6, 7
67     beq t3, t6, AND          # AND (funct3=7)
68     jr ra
69

```

Figura 2.34. Righe 26-68 di `simula_R.S`, Sezione di decodifica e caricamento operandi

Nella prima fase del codice viene effettuata la decodifica dell'istruzione, dalla quale vengono estratti i campi `rs1`, `rs2`, `rd`, `func3` e `func7`. Successivamente, i valori contenuti nei registri `rs1` e `rs2` vengono caricati dalla memoria dei registri virtuali.

In base al valore del campo `func3`, il flusso di esecuzione del programma viene indirizzato verso una specifica etichetta, ognuna delle quali implementa una diversa operazione aritmetico-logica (ad esempio somma, sottrazione, shift, operazioni logiche o confronti).

```

106      XOR:
107      xor s7, a3, a4
108      slli t4, t4, 2
109      add t1, t4, t1
110      sw  s7, 0(t1)
111      j  vai_avanti_con_pc
112

```

Figura 2.35. Esempio di operazione aritmetica

```

141
142  vai_avanti_con_pc:
143      addi s2, s2, 8
144      jr  ra
145

```

Figura 2.36. Scrittura del risultato nel registro di destinazione

Nella fase finale viene eseguita l'operazione specificata dall'istruzione: il risultato viene calcolato e successivamente salvato nel registro di destinazione **rd** simulato all'interno della memoria dei registri virtuali.

Infine, il PC simulato viene incrementato per puntare all'istruzione successiva. Terminata l'esecuzione il programma torna al chiamante ovvero al modulo `controllo_dinamico_loop.S`.

2.5.7 simula_S.S

```

2
3  # IL PROGRAMMA SIMULA LE SEGUENTI OPERAZIONI:
4
5  #-----#
6  # sb Store Byte S 0100011 0x0 M[rs1+imm][0:7] = rs2[0:7]
7  # sh Store Half S 0100011 0x1 M[rs1+imm][0:15] = rs2[0:15]
8  # sw Store Word S 0100011 0x2 M[rs1+imm][0:31] = rs2[0:31]
9  #-----#
10
11 # |imm| rs2 | rs1 | funct3 | imm | opcode | (S-type)
12

```

Figura 2.37. Righe 3-11 di `simula_S.S`

Il modulo `simula_S` è responsabile della simulazione delle istruzioni di tipo S. Queste sono operazioni di allocazione dati nella memoria. vengono salvati i dati contenuti in **rs2** nell'indirizzo di memoria **rs2 + imm**, ci sono vari tipi di funzioni di stoccaggio dati, la specifica operazione da eseguire è determinata dal campo **func3**.

```
16
17 .text
18 simula_S:
19     # Estrazione campi
20     srli t1, t6, 15
21     andi t1, t1, 31          # t1 = rs1
22
23     srli t2, t6, 20
24     andi t2, t2, 31          # t2 = rs2
25
26     srli t3, t6, 12
27     andi t3, t3, 7           # t3 = funct3
28
29     srli a6, t6, 7
30     andi a6, a6, 31          # a6 = imm[4:0]
31
32     srli t4, t6, 25
33     andi t4, t4, 127         # t4 = imm[11:5]
34
35     slli t4, t4, 5
36     or s9, t4, a6
37     slli s9, s9, 20
38     srai s9, s9, 20          # s9 = imm
39
```

Figura 2.38. Righe 17-38 di `simula_S.S`

Nella prima sezione del codice viene eseguita la decodifica dell'istruzione in ingresso, dalla quale vengono estratti i campi `rs1`, `rs2`, `func3`. In seguito, viene effettuata la ricostruzione del campo `imm`. dato che in questo formato di istruzioni il campo `imm` è diviso in più parti nella word d'istruzione vengono estratti separatamente i vari pezzi (bit di segno, bit centrali e bit meno significativi) e successivamente ricostruito combinando questi pezzi con varie operazioni logiche.

```

39
40     # Caricamento valori registri virtuali
41
42     slli t1, t1, 2          # offset rs1
43     add  t1, t1, a5
44     lw   a3, 0(t1)         # a3 = valore rs1
45
46     slli t2, t2, 2          # offset rs2
47     add  t2, t2, a5
48     lw   a4, 0(t2)         # a4 = valore rs2
49
50     add t5, a3, s9          # t5 = rs1 + imm
51     add t5, t5, s8          # t5 = t5 + indirizzo base memoria virtuale
52
53

```

Figura 2.39. Righe 40-51 di simula_S.S

In t5 viene salvato l'indirizzo dove salvare i dati, questo indirizzo è ottenuto sommando al valore di rs1 l'indirizzo di offset + il campo immediato.

Successivamente, i valori contenuti nei registri rs1 e rs2 vengono caricati dalla memoria dei registri virtuali. In base al valore del campo func3, il flusso di esecuzione viene indirizzato verso una specifica etichetta, ognuna delle quali implementa una diversa operazione di allocazione dati (byte, word, half)

```

53
54     beq t3, zero, SB      # SB (funct3=0)
55     li  t1, 1
56     beq t3, t1, SH       # SH (funct3=1)
57     li  t1, 2
58     beq t3, t1, SW       # SW (funct3=2)
59     jr  ra
60
61     SB:
62     |   sb a4, 0(t5)
63     |   j  vai_avanti_con_pc
64     SH:
65     |   |   sh a4, 0(t5)
66     |   |   j  vai_avanti_con_pc
67     SW:
68     |   |   sw a4, 0(t5)
69     |   |   j  vai_avanti_con_pc
70
71     vai_avanti_con_pc:
72     |   addi s2, s2, 8
73     |   jr  ra
74

```

Figura 2.40. Righe 54-73 di simula_S.S

Nella fase finale viene eseguita l'operazione specificata dall'istruzione (campo func3), i dati vengono salvati nella memoria simulata (spostata con offset). Infine, il PC simulato viene incrementato per puntare all'istruzione successiva. Terminata l'esecuzione il programma torna al chiamante ovvero al modulo `controllo_dinamico_loop.S`.

2.5.8 simula_I.S

```

2
3 # IL PROGRAMMA SIMULA LE SEGUENTI:
4
5 #-----#
6 # addi ADD Immediate I 0010011 0x0 rd = rs1 + imm
7 # xori XOR Immediate I 0010011 0x4 rd = rs1 o imm
8 # ori OR Immediate I 0010011 0x6 rd = rs1 | imm
9 # andi AND Immediate I 0010011 0x7 rd = rs1 & imm
10 # slli Shift Left Logical Imm I 0010011 0x1 imm[5:11]=0x00 rd = rs1 << imm[0:4]
11 # srli Shift Right Logical Imm I 0010011 0x5 imm[5:11]=0x00 rd = rs1 >> imm[0:4]
12 # srai Shift Right Arith Imm I 0010011 0x5 imm[5:11]=0x20 rd = rs1 >> imm[0:4] msb-extends
13 # slti Set Less Than Imm I 0010011 0x2 rd = (rs1 < imm)?1:0
14 # sltiu Set Less Than Imm (U) I 0010011 0x3 rd = (rs1 < imm)?1:0 zero-extends
15 #-----#
16 # lb Load Byte I 0000011 0x0 rd = M[rs1+imm][0:7]
17 # lh Load Half I 0000011 0x1 rd = M[rs1+imm][0:15]
18 # lw Load Word I 0000011 0x2 rd = M[rs1+imm][0:31]
19 # lbu Load Byte (U) I 0000011 0x4 rd = M[rs1+imm][0:7] zero-extends
20 # lhu Load Half (U) I 0000011 0x5 rd = M[rs1+imm][0:15] zero-extends
21 #-----#
22 # jalr Jump And Link Reg I 1100111 0x0 rd = PC+4; PC = rs1 + imm
23 #-----#
24 # ecall Environment Call I 1110011 0x0 imm=0x0 Transfer control to OS
25 # ebreak Environment Break I 1110011 0x0 imm=0x1 Transfer control to debugger
26 #-----#
27
28 # imm | rs1 | funct3 | rd | opcode (I-type)
29

```

Figura 2.41. Righe 3-28 di `simula_I.S`

Il modulo `simula_I` è responsabile della simulazione delle istruzioni di tipo I cioè operazioni aritmetico-logiche tra un registro `rs1` e un valore immediato `imm`, oppure operazioni di accesso alla memoria, salto o chiamate di sistema (*che tuttavia non verranno usate*). Il risultato dell'operazione viene salvato nel registro di destinazione `rd`. Il comportamento dell'istruzione dipende dal campo `opcode` e dal campo `func3`.

```

32  simula_I:
33      srli t1, t6, 15
34      andi t1, t1, 31          # t1 = rs1
35
36      srli t3, t6, 12
37      andi t3, t3, 7          # t3 = funct3
38
39      srli t4, t6, 7
40      andi t4, t4, 31          # t4 = rd
41
42      srli t5, t6, 20          # immediato
43      slli t5, t5, 20
44      srai t5, t5, 20
45
46      slli t1, t1, 2            # offset rs1
47      add t1, t1, a5
48      lw a3, 0(t1)             # a3 = valore rs1
49

```

Figura 2.42. Righe 32-48 di simula_I.S

```

49
50      # Confronti con opcode (#s10 opcode fisso)
51      li s9,19
52      beq s10, s9, vai_simula_ALGI
53      li s9,3
54      beq s10, s9, vai_simula_LOAD
55      li s9,103
56      beq s10, s9, vai_simula_JALR
57      li s9,115
58      beq s10, s9, vai_simula_SYST
59

```

Figura 2.43. Righe 50-58 di simula_I.S

Nella prima fase del codice viene effettuata la decodifica dell'istruzione, dalla quale vengono estratti i campi **rs1**, **rd**, **func3** e l'immediato **imm**, il valore contenuto nel registro **rs1** viene caricato dalla memoria dei registri virtuali.

Nel caso delle istruzioni aritmetico-logiche immediate, viene effettuata un'ulteriore selezione in base al campo **func3**. Ogni etichetta implementa una specifica operazione. Il risultato viene poi salvato nel registro di destinazione **rd**.

Se invece la word non rappresenta un'istruzione aritmetico-logico immediata avremo una tra le seguenti situazioni:

- **Istruzione di load:** viene calcolato l'indirizzo di memoria come somma tra `rs1` e `imm`. Il dato viene poi letto dalla memoria virtuale e salvato nel registro `rd` simulato.
- **Istruzione JALR:** viene salvato nel registro `rd` simulato l'indirizzo di ritorno (`PC + 4`), mentre il PC simulato viene aggiornato con il valore `rs1 + imm`.
- **Istruzione di sistema:** il flusso viene deviato in base al valore dell'immediato. Nel caso di `EBREAK` l'esecuzione termina, mentre `ECALL` comporta un semplice avanzamento del programma. *Questo tipo di funzioni non verranno usate negli esercizi.*
- **Aggiornamento del PC e ritorno:** per tutte le istruzioni che non modificano direttamente il flusso di controllo, il PC simulato viene aumentato di una word per puntare all'operazione subito successiva a quella che è stata appena simulata. Terminata l'esecuzione, il flusso torna al modulo `controllo_dinamico_loop.S`.

2.5.9 simula_J.S

```

2
3  # IL PROGRAMMA SIMULA LE SEGUENTI OPERAZIONI:
4
5  #-----#
6  # jal Jump And Link J 1101111 rd = PC+4; PC += imm
7  #-----#
8  # imm | rd | opcode (J-type)
9

```

Figura 2.44. Righe 3-8 di `simula_J.S`

Il modulo `simula_J` è responsabile della simulazione delle istruzioni di tipo J. In particolare, implementa l'istruzione JAL (Jump And Link), che consente di effettuare un salto salvando contemporaneamente l'indirizzo di ritorno nel registro di destinazione `rd`.

```

9
10 .globl simula_J
11
12 .text
13 simula_J:
14     srli t4, t6, 7
15     andi t4, t4, 31      # t4 = rd
16
17     addi a7, s2, 8      # pc + 4
18
19     slli t4, t4, 2      # indirizzo ai registri virtuali
20     add  t4, t4, a5
21     sw   a7, 0(t4)
22

```

Figura 2.45. Righe 19-21 di `simula_J.S`

Nella prima fase del codice viene estratto il campo `rd` dall'istruzione. Successivamente, viene calcolato il valore `PC + 4`, che rappresenta l'indirizzo dell'istruzione subito successiva a quella in corso di simulazione, questo valore viene salvato nel registro `rd` all'interno della memoria dei registri virtuali per essere utilizzato in seguito.

```

25
26  # Bit 20 (Segno)
27      srli t1, t6, 31
28      slli t1, t1, 20
29
30      # Bit 19:12
31      lui  t2, 0x000FF          # Maschera per bit 19:12
32      and  t2, t2, t6
33
34      # Bit 11
35      srli t3, t6, 20
36      andi t3, t3, 1
37      slli t3, t3, 11
38
39      # Bit 10:1
40      srli t5, t6, 21
41      andi t5, t5, 0x3FF
42      slli t5, t5, 1
43
44      # Unione
45      or   t1, t1, t2
46      or   t1, t1, t3
47      or   t1, t1, t5          # t1 ha l'immediato a 21 bit
48
49      # Estensione del segno (da 21 a 32 bit)
50      slli t1, t1, 11
51      srai t1, t1, 11
52

```

Figura 2.46. Righe 26-51 di `simula_J.S`

In seguito, viene effettuata la ricostruzione del campo `imm`. dato che in questo formato di istruzioni il campo `imm` è diviso in più parti nella word vengono estratti separatamente i vari pezzi (bit di segno, bit centrali e bit meno significativi) e successivamente ricostruito combinando questi pezzi con varie operazioni logiche.

Una volta ricostruito, l'immediato viene esteso con segno per ottenere il valore finale a 32 bit.

```

53  #-----
54      li s11,2
55      mul t1, t1, s11
56      add s2, s2, t1          # nuovo PC (simulato) = PC (simulato) + immediato
57      jr ra
58

```

Figura 2.47. Righe 54-57 di `simula_J.S`

Infine, l'immediato viene sommato al PC simulato, simulando così il salto (`PC = PC + imm`).

Terminata la simulazione dell'istruzione, il controllo viene restituito al chiamante `controllo_dinamico_loop.S`.

2.5.10 simula_U.S

```

2
3 # IL PROGRAMMA SIMULA LE SEGUENTI OPERAZIONI:
4 #-----#
5 # lui    Load Upper Imm      U 0110111   rd = imm << 12
6 # auipc  Add Upper Imm to PC U 0010111   rd = PC_simulato + (imm << 12)
7 #-----#
8 # imm[31:12] | rd | opcode   (U-type)

```

Figura 2.48. Righe 3-8 di simula_U.S

Il modulo `simula_U` è responsabile della simulazione delle istruzioni di tipo U, in particolare LUI (Load Upper Immediate) e AUIPC (Add Upper Immediate to PC).

```

11
12 .text
13 simula_U:
14
15     srli s9, t6, 7
16     andi s9, s9, 31           # s9 = rd
17
18     lui  t2, 0xFFFFF
19     and  t2, t6, t2           # t2 = imm << 12
20
21     # capisco che op è
22     li   t4, 55               # LUI
23     beq  s10, t4, LUI
24
25     li   t4, 23               # AUIPC
26     beq  s10, t4, AUIPC
27
28     jr  ra
29

```

Figura 2.49. Righe 12-28 di simula_U.S

Nella fase iniziale del codice viene estratto il campo `rd` dall'istruzione tramite shift e mascheramento. In parallelo, viene ricostruito l'immediato U-type: i 20 bit più significativi vengono isolati dall'istruzione e successivamente spostati di 12 posizioni a sinistra, ottenendo il valore `imm << 12`.

Il modulo identifica il tipo di istruzione confrontando il codice operativo (`s10`) con i valori noti:

- 55 → LUI
- 23 → AUIPC

In base al risultato del confronto, il flusso viene indirizzato alla relativa sezione di esecuzione.

```

29
30  LUI:
31      # salvo in s11 per non creare sovrapposizioni dopo
32      mv    s11, t2
33      j     salva_e_proseguì
34
35  AUIPC:
36
37      li s9, 4096
38      add t0, s8, s9
39      sub t1, s2, t0
40      srai t1, t1, 1
41
42      lui  t0, 0x00400
43      add t1, t1, t0
44
45      add s11, t1, t2
46      j     salva_e_proseguì
47
48  salva_e_proseguì:
49      slli t0, s9, 2
50      add t0, t0, a5      # Indirizzo ai registri virtuali
51      sw  s11, 0(t0)
52
53      j vai_avanti_con_pc
54
55  vai_avanti_con_pc:
56      addi s2, s2, 8
57      jr  ra
58

```

Figura 2.50. Righe 30-57 di `simula_U.S`

Nel caso dell'istruzione LUI, il valore dell'immediato già scalato ($imm \ll 12$) viene direttamente copiato nel registro temporaneo `s11`. Questo valore rappresenta il contenuto da scrivere nel registro di destinazione `rd`.

Nel caso dell'istruzione AUIPC, il comportamento è differente: il valore immediato viene sommato al PC simulato (`s2`). Il risultato intermedio viene elaborato attraverso alcune operazioni aritmetiche e di aggiustamento, fino a ottenere il valore finale che rappresenta:

$$rd = PC_{simulato} + (imm \ll 12)$$

Il risultato viene infine salvato in `s11`.

Una volta calcolato il valore finale, il modulo calcola l'indirizzo del registro virtuale corrispondente a `rd` tramite:

- shift del valore di `rd`
- somma con la base della memoria registri (`a5`)

Il valore contenuto in `s11` viene quindi scritto in memoria. Infine, il Program Counter simulato viene aggiornato avanzando di una word ($PC = PC + 8$), e il flusso torna al modulo chiamante `controllo_dinamico_loop.S`.

2.6 Il diagramma UML di flusso del programma

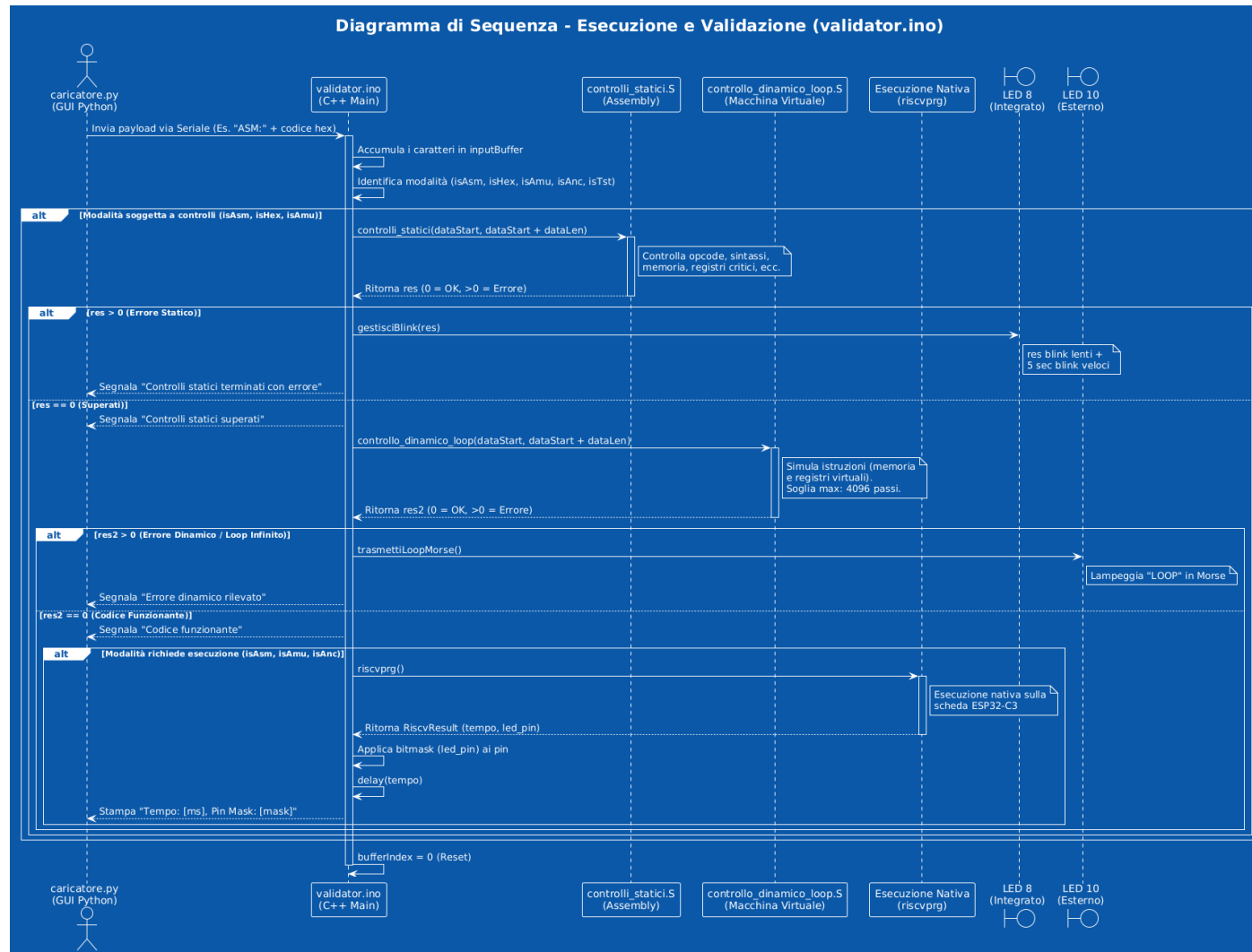


Figura 2.51. Diagramma UML di flusso generale (Validator.ino)

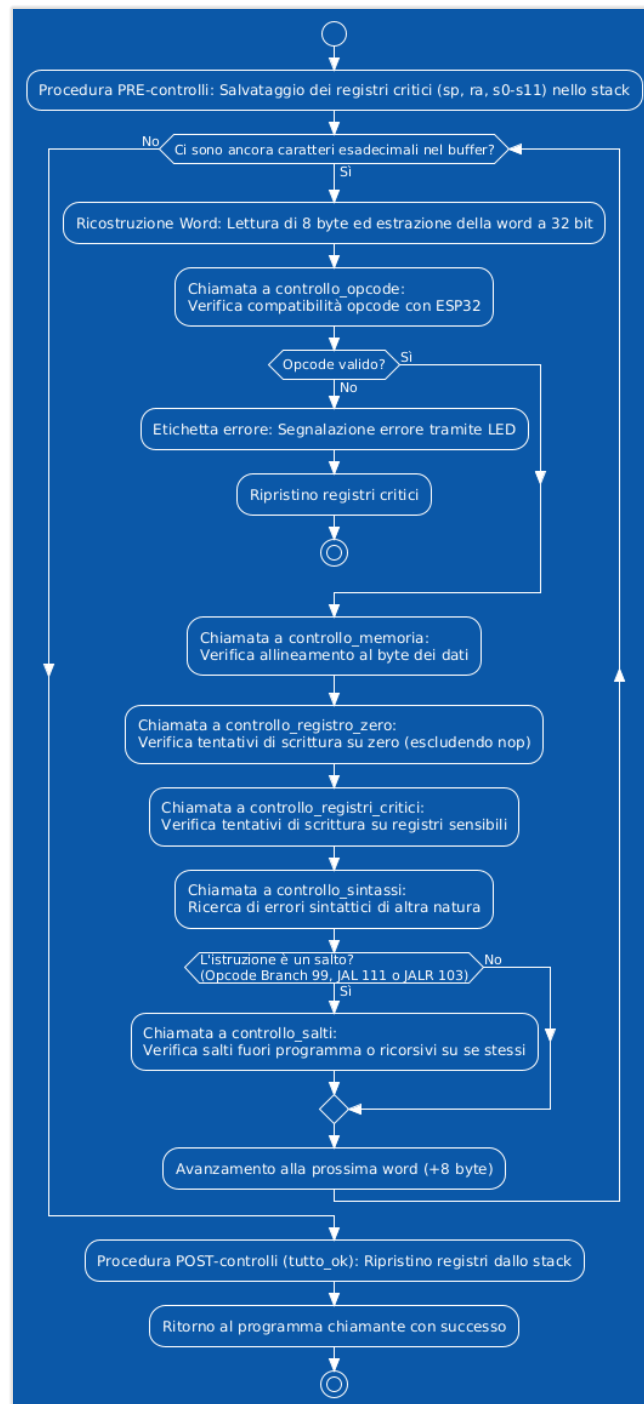


Figura 2.52. Diagramma UML di flusso di controlli_statici.S

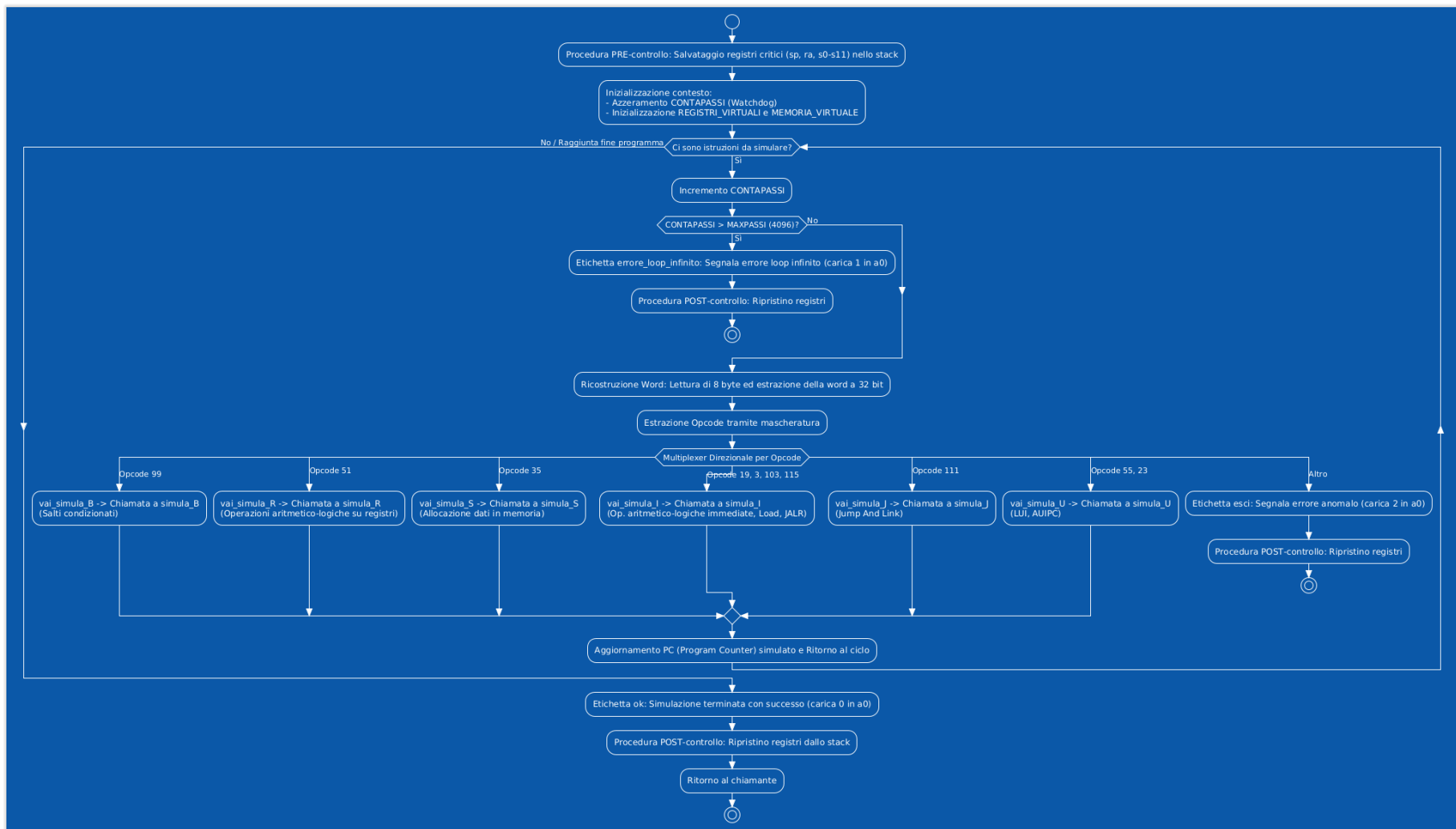


Figura 2.53. Diagramma UML di flusso di controllo_dinamico_loop.S

2.7 La repository GitHub

The screenshot shows the GitHub repository page for 'RISC-Visore' by user 'daromacarlo'. The repository is public and has 18 commits. The main branch is 'main'. The repository contains a README.md file, a .gitattributes file, a test directory, a validator directory, and a documentation directory. The README.md file is selected and shows the following content:

platform ESP32-C3 architecture RISC-V language Assembly

Semplice tool per ESP32-C3 SuperMini con architettura Risc-V volto ad agevolare l'apprendimento dell'architettura stessa. Il sistema permette la scrittura, compilazione ed esecuzione di codice Assembly direttamente sul microcontrollore. Il software offre varie modalità di utilizzo ed è utile per validare vari tipi di esercizi, concepiti per supportare chiunque desideri approcciare lo studio di RISC-V.

The right sidebar shows the 'About' section with no description, website, or topics provided. It also shows the 'Releases' section with no releases published and the 'Packages' section with no packages published. The 'Contributors' section shows one contributor, 'daromacarlo' (Carlo Da Roma). The 'Languages' section shows a bar chart with the following data:

Language	Percentage
Makefile	84.8%
Assembly	11.6%
Python	2.4%
C++	1.0%

Tutto il codice descritto è visionabile e scaricabile al seguente link: [RISC-Visore](#)



Capitolo 3

Gli esercizi

Segue una sezione che illustra le principali categorie di esercizi possibili.

3.1 Tipologia 1.

Come prima categoria di esercizi è possibile chiedere a chi vuole imparare il linguaggio di scrivere semplice codice assembly RISC-V.

L'obiettivo è caricare nel registro `a0` un valore che rappresenti la durata di accensione di un LED, mentre nel registro `a1` deve essere caricato un array di bit che specifica i LED da attivare.

Il codice prodotto dovrà essere caricato tramite MODALITÀ ASSEMBLY in `caricatore.py` e l'esito è visualizzabile da console e interfaccia LED

Traccia1.

Scrivere un programma in assembly RISC-V che, dato un vettore di word, faccia accendere il LED x per un tempo pari a $y + 1000$ ms, dove:

- x è il valore massimo nel vettore
- y è il valore minimo nel vettore

Vettore:

```
.word 15, 20, 42, 90, 88, 1, 40
```

Possibile soluzione:

```
.globl main

.data
vettore:    .word 15, 20, 42, 90, 88, 1, 40

.text
main:

    la t0, vettore      # puntatore al vettore
    li t1, 7            # numero elementi

    lw a0, 0(t0)        # massimo iniziale
    lw a1, 0(t0)        # minimo iniziale

    addi t0, t0, 4
    addi t1, t1, -1

loop:
    blez t1, END
```

```

    lw t2, 0(t0)

    ble t2, a0, check_min
    mv a0, t2          # aggiorna massimo
    j skip_update

check_min:
    bge t2, a1, skip_update
    mv a1, t2          # aggiorna minimo

skip_update:
    addi t0, t0, 4
    addi t1, t1, -1
    j loop

END:
    addi a0, a0, 1000  # tempo = minimo + 1000 ms

```

3.2 Tipologia 1. con test multipli.

Può essere chiesto di scrivere semplice codice assembly RISC-V.

L'obiettivo è caricare nel registro `a0` un valore che rappresenti la durata di accensione di un LED, mentre nel registro `a1` deve essere caricato un array di bit che specifica i LED da attivare.

Questo codice viene testato con `.data` diversi, Il codice prodotto dovrà essere caricato tramite MODALITÀ MULTIPLA insieme ai vari `.data` con cui si vuole provare il codice in `caricatore.py` e l'esito è visualizzabile da console e interfaccia LED

Traccia1.

Scrivere un programma in assembly RISC-V che, dato un vettore di word, faccia accendere il LED x per un tempo pari a $y + 1000$ ms, dove:

- x è il valore massimo nel vettore
- y è il valore minimo nel vettore

Input:

```

Vettore: .word 15, 20, 42, 90, 88, 1, 40;
Vettore: .word 15, 20, 42, 900, 88, 1, 40;
Vettore: .word 15, 20, 42, 9000, 88, 1, 40

```

Possibile soluzione:

```

.globl main

.data

.text
main:

    la t0, vettore      # puntatore al vettore
    li t1, 7            # numero elementi

    lw a0, 0(t0)        # massimo iniziale
    lw a1, 0(t0)        # minimo iniziale

```

```

        addi t0, t0, 4
        addi t1, t1, -1

loop:
    blez t1, END

    lw t2, 0(t0)

    ble t2, a0, check_min
    mv a0, t2          # aggiorna massimo
    j skip_update

check_min:
    bge t2, a1, skip_update
    mv a1, t2          # aggiorna minimo

skip_update:
    addi t0, t0, 4
    addi t1, t1, -1
    j loop

END:
    addi a0, a0, 1000  # tempo = minimo + 1000 ms

```

3.3 Tipologia 2.

Come secondo tipo di esercizio può essere proposto di modificare il `.data` per ottenere un certo effetto sui led della scheda, la difficoltà dell'esercizio risiede nella comprensione del codice assembly fornito. Il codice modificato dovrà essere caricato tramite MODALITÀ ASSEMBLY in `caricatore.py` e l'esito è visualizzabile da console e interfaccia LED

Traccia 1.

Modifica la sezione `.data` del seguente programma in assembly RISC-V in modo tale che venga acceso il led 9 della ESP per 1005ms. NON PUOI MODIFICARE IL `.text` E IL VETTORE IN INPUT DEVE RIMANERE DELLA STESSA LUNGHEZZA.

```

.globl main

.data
vettore:    .word 15, 20, 42, 90, 88, 10, 40

.text
main:

    la t0, vettore
    li t1, 7

    lw a0, 0(t0)
    lw a1, 0(t0)

    addi t0, t0, 4
    addi t1, t1, -1

loop:
    blez t1, END

    lw t2, 0(t0)

```

```

        ble t2, a0, check_min
        mv a1, t2
        j skip_update

check_min:
        bge t2, a0, skip_update
        mv a0, t2

skip_update:
        addi t0, t0, 4
        addi t1, t1, -1
        j loop

END:
        addi a0, a0, 1000

```

Possibile soluzione:

```

.globl main

.data
vettore:    .word 5, 512, 5, 6, 6, 6, 7

.text
main:

        la t0, vettore
        li t1, 7

        lw a0, 0(t0)
        lw a1, 0(t0)

        addi t0, t0, 4
        addi t1, t1, -1

loop:
        blez t1, END

        lw t2, 0(t0)

        ble t2, a0, check_min
        mv a0, t2
        j skip_update

check_min:
        bge t2, a1, skip_update
        mv a1, t2

skip_update:
        addi t0, t0, 4
        addi t1, t1, -1
        j loop

END:
        addi a0, a0, 1000

```

3.4 Tipologia 3.

Come terzo tipo di esercizio può essere proposto di modificare il `text` di un programma Assembly RISC-V per evitare che questo vada in loop infinito quando eseguito. La difficoltà di tale tipologia di esercizi sta nel riuscire a modificare stando attenti alle limitazioni imposte dalla consegna. Possono essere posti limiti su quante operazioni è possibile modificare, eliminare o aggiungere. Il codice modificato dovrà essere caricato tramite MODALITÀ ASSEMBLY in `caricatore.py` e l'esito è visualizzabile da console e interfaccia LED

Traccia 1.

Si consideri il seguente programma in assembly RISC-V:

```
.globl main

.text
main:
j step1

END:
    j REAL_END
    j step1
    j step2

step1:
    j step2

step2:
    j check
    j END

step3:
    j check

step4:

check:
    beq zero, zero, loop

loop:
    j again

again:
    j step1

REAL_END:
    addi a0, a0, 1000
    addi a1, a1, 1024
```

Il programma attualmente entra in un loop infinito a causa della struttura dei salti.

Modificare il programma in maniera minimale, cioè aggiungendo, rimuovendo o modificando il minor numero di operazioni possibili e in modo tale che:

- Il flusso venga eseguito una sola volta passando attraverso le varie etichette `step1`, `step2`, `step3`, `check`
- Il programma termini correttamente all'etichetta `REAL_END`

- Non venga utilizzato alcun loop infinito o salto ciclico
- È consentito modificare, aggiungere o rimuovere solo le istruzioni di salto `j`, `beq`
- Non è consentito aggiungere istruzioni di memoria o modificare la logica aritmetica
- Il numero di etichette deve rimanere invariato

IL NUMERO MASSIMO DI MODIFICHE CHE PUOI EFFETTUARE È 5

Possibile soluzione:

```
.globl main

.text
main:
j step1

END:
    j REAL_END
    j step1
    j step2

step1:
    j step2

step2:
    j check
    j END

step3:
    j check

step4:
    J loop
check:
    j step3
    beq zero, zero, loop

loop:
    j again

again:
    j END
    j step1

REAL_END:
    addi a0, a0, 1000
    addi a1, a1, 1024
```

3.5 Tipologia 4.

Come quarto tipo di esercizio può essere proposto di modificare il digest di un programma Assembly RISC-V per evitare che questo presenti errori di vario tipo. La difficoltà di tale tipologia di esercizi sta nel riuscire a modificare il programma byte per byte, è questa un'operazione molto di basso livello. Il digest modificato dovrà essere caricato tramite MODALITÀ IMMEDIATA in `caricatore.py` e l'esito è visualizzabile solo da interfaccia LED

Traccia 1.

Si consideri il seguente programma:

```
.globl main

.text

    main:
    li a0, 6000
    li a1, 10
```

il cui digest è

37150000130505779305a00000

Il programma digest presentato attualmente presenta errori, identificarli tramite l'interfaccia LED e risolverli.

Si ricorda che la formattazione è `Little endian`

Modificare il programma in maniera minimale, cioè aggiungendo, rimuovendo o modificando il minor numero di operazioni possibili, Risolvere i problemi in ordine in base a come vengono segnalati.

IL NUMERO MASSIMO DI MODIFICHE CHE PUOI EFFETTUARE È 1

Possibile soluzione: La scheda segnala un errore di tipo 2, infatti il led integrato lampeggia 2 volte lentamente e poi lampeggia per 5 secondi velocemente, significa che siamo davanti ad un errore di word intere, dobbiamo quindi eliminare dei bit superflui.

Il digest modificato corretto sarà:

37150000130505779305a000

Per modificare gli hex è consigliato scaricare [Free Hex Editor Neo](#)

3.6 L'eserciziario

Nella sezione RISC-Visore/validator/test sono disponibili svariati esercizi divisi per categoria.



Capitolo 4

Conclusioni

4.1 Difficoltà incontrate

Durante lo sviluppo del progetto sono emerse diverse difficoltà. In particolare, la gestione diretta dei LED tramite codice assembly si è rivelata complessa ed imprecisa, richiede la temporizzazione tramite clock della scheda e molte righe di codice, per questo nel mio programma ho preferito gestire i LED nello strato più superficiale.

Un'ulteriore criticità è stata la gestione del puntatore al programma caricato riscv1.S. È possibile su queste schede puntare un programma e richiamarlo dinamicamente ma la gestione di questo puntatore è molto articolata quindi ho preferito ricompilare ogni volta che un nuovo programma assembly viene caricato

4.2 Sviluppi possibili

Il codice sviluppato rappresenta un punto di partenza per la realizzazione di una vera e propria macchina virtuale RISC-V. Un'importante miglioria sarebbe l'integrazione della sezione `.data`, che permetterebbe la gestione di variabili statiche e strutture dati direttamente in memoria, avvicinando ulteriormente il sistema al comportamento di un ambiente di esecuzione reale.

Bibliografia

- [1] R. Tartaglione, *Generazione e correzione automatizzata di esercizi di sistemi di memoria cache*, Strumenti didattici per l'Architettura degli Elaboratori.
- [2] M. Ziegler, *Simulatore MIPS orientato all'analisi e all'apprendimento delle dinamiche del pipelining*, Software Didattico Interattivo.
- [3] D. A. Patterson e J. L. Hennessy, *RISC-V as the New Standard for Computer Architecture Education*, ACM Digital Library.
<https://dl.acm.org/doi/abs/10.1145/3445034.3460508>
- [4] IEEE Xplore, *Cloud-Based Web Simulators for Computer Architecture Instruction*.
<https://ieeexplore.ieee.org/abstract/document/9071132>
- [5] ScienceDirect, *Visual Simulation Frameworks for RISC-V Pipeline and Datapath Analysis*, Journal of Systems Architecture.
<https://www.sciencedirect.com/science/article/pii/S1383762122002429>
- [6] WOSET Workshop, *The Educational RISC-V Micro: Designing Custom Cores for FPGA Synthesizable Laboratories*.
https://woset-workshop.github.io/PDFs/2024/9_The_Educational_RISC_V_Micro.pdf
- [7] IATED Digital Library, *FPGA-Based RISC-V Core Implementation in Undergraduate Computer Engineering Courses*.
<https://library.iated.org/view/SPRANG2024FPG>
- [8] MDPI Computers, *Integrating Real-World RISC-V Microcontrollers in IoT and Edge Computing Higher Education*.
<https://www.mdpi.com/2079-9268/12/3/45>
- [9] IEEE Xplore, *Hands-on Embedded Systems Labs: Shifting from Pure Simulation to Affordable Commercial Hardware*.
<https://ieeexplore.ieee.org/abstract/document/10540579>